UNDERGRADUATE LECTURE

Advanced Tree Models

From Decision Trees to XGBoost, LightGBM and CatBoost

Decision trees · Random forests · AdaBoost · Gradient boosting

A visual tour with no equations: every idea is explained with a picture.
All charts are drawn from real model fits on small simulated data sets.

Three ideas carry the whole lecture

Keep these in mind; every later slide is a detail of one of them

1

A tree is a list of yes/no questions

One tree is easy to read but unstable: small changes in data change the tree.

2

Averaging many trees reduces variance

Bagging and random forests grow deep trees in parallel and average them.

3

Boosting adds small trees that fix errors

AdaBoost, gradient boosting, XGBoost, LightGBM and CatBoost build trees in sequence.

Single tree → many trees in parallel → many trees in sequence

The lecture moves in five steps

Each step builds on the one before it

1 Tree

building block

Simple yes/no questions. Unstable.

CART

2 Forest

in parallel

Average many deep trees.

Random forest

3 Boosting

in sequence

Small trees fix earlier errors.

AdaBoost, GBM

4 GBDT tools

regularized

Penalties, missing values, categories.

XGBoost and co.

5 Explain

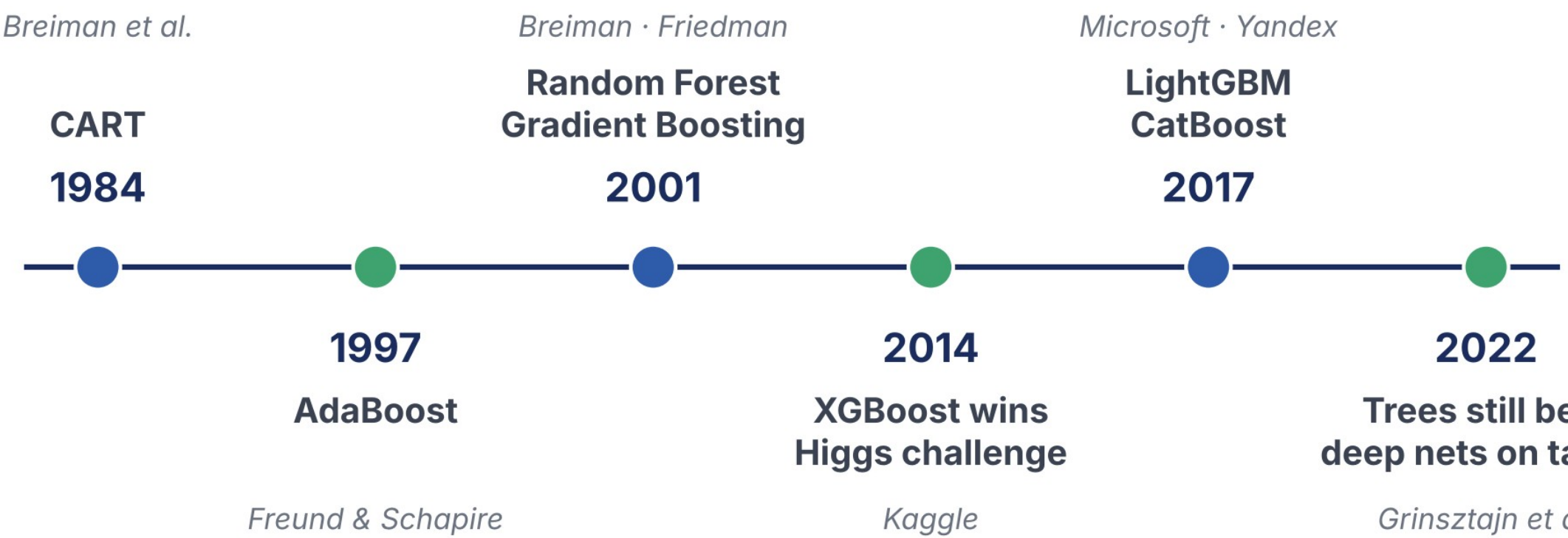
open the box

Which inputs matter, and why?

SHAP

Forty years of tree models, from CART to LightGBM

Key papers and events; the 2022 study found trees still beat deep nets on most tabular benchmarks



01

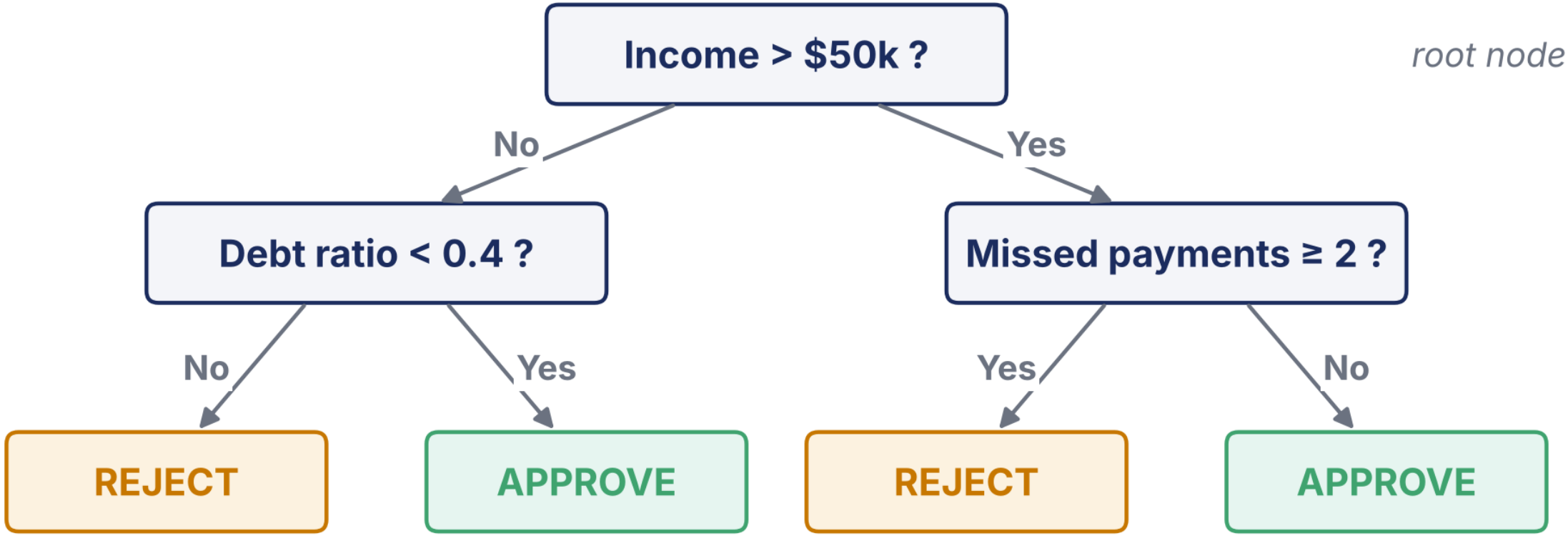
PART 1 · DECISION TREES

Decision Trees: The Building Block

How one tree asks questions, chooses splits and overfits

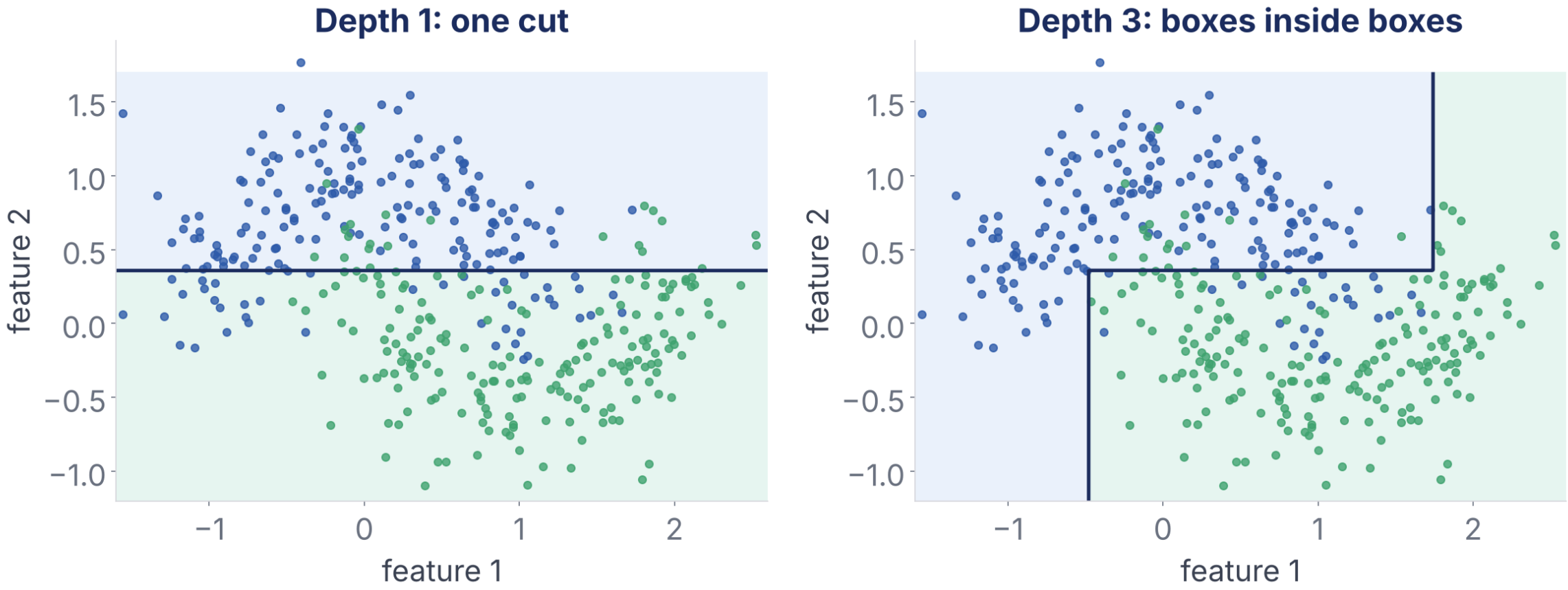
A decision tree is a sequence of yes/no questions

Toy loan example: start at the root, follow the answers, read the prediction at a leaf



Every split cuts the feature space with a straight line

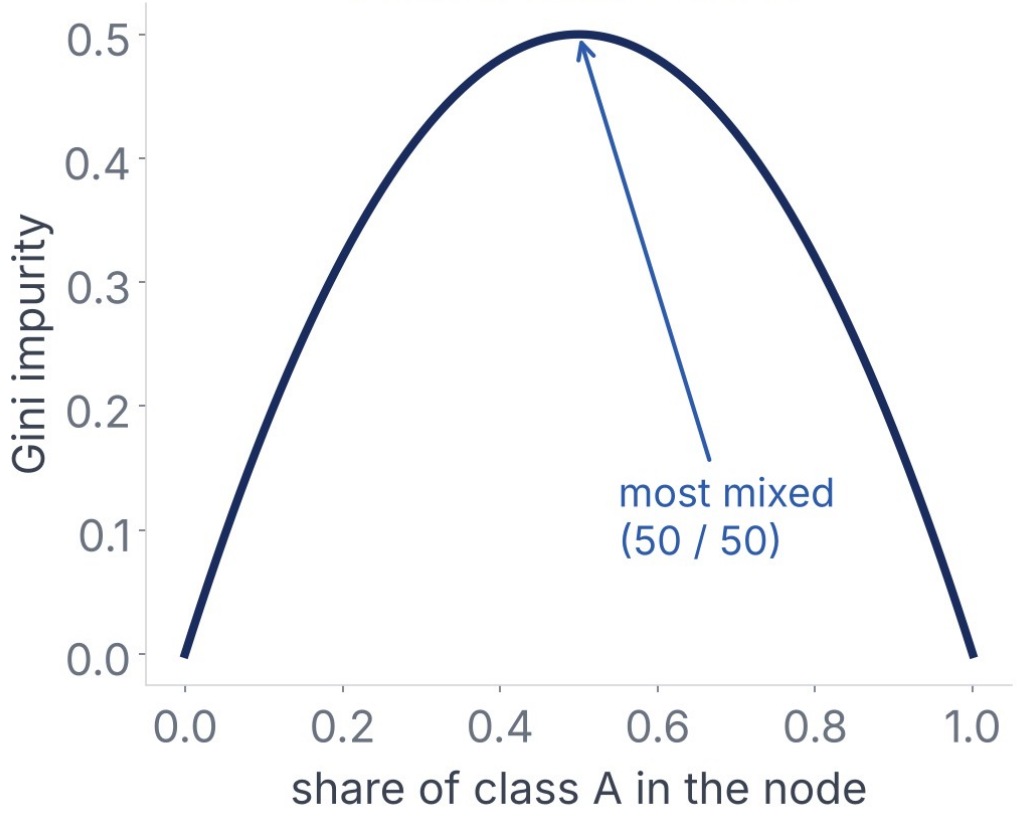
Two-moons data: deeper trees stack axis-parallel cuts into rectangles



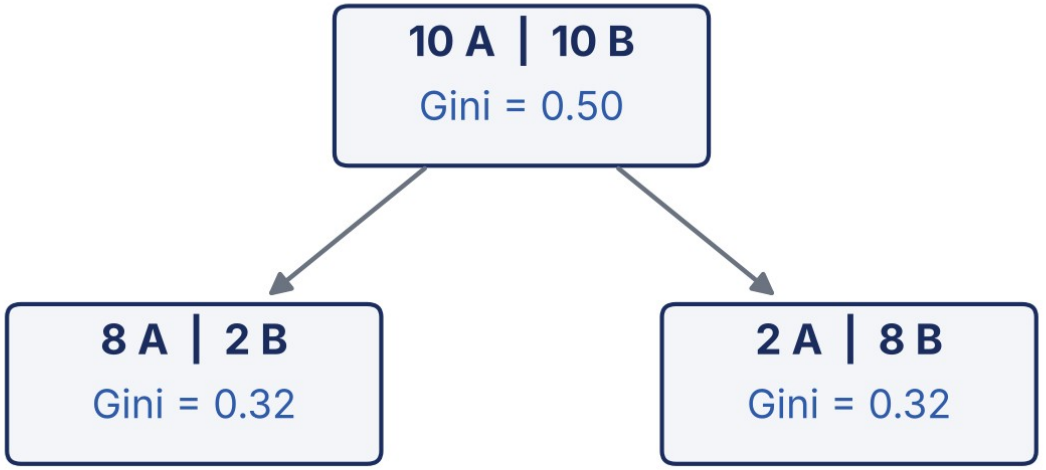
Splits are chosen to make child nodes purer

Gini impurity measures how mixed a node is: 0 when pure, largest at a 50/50 mix

Pure nodes score 0



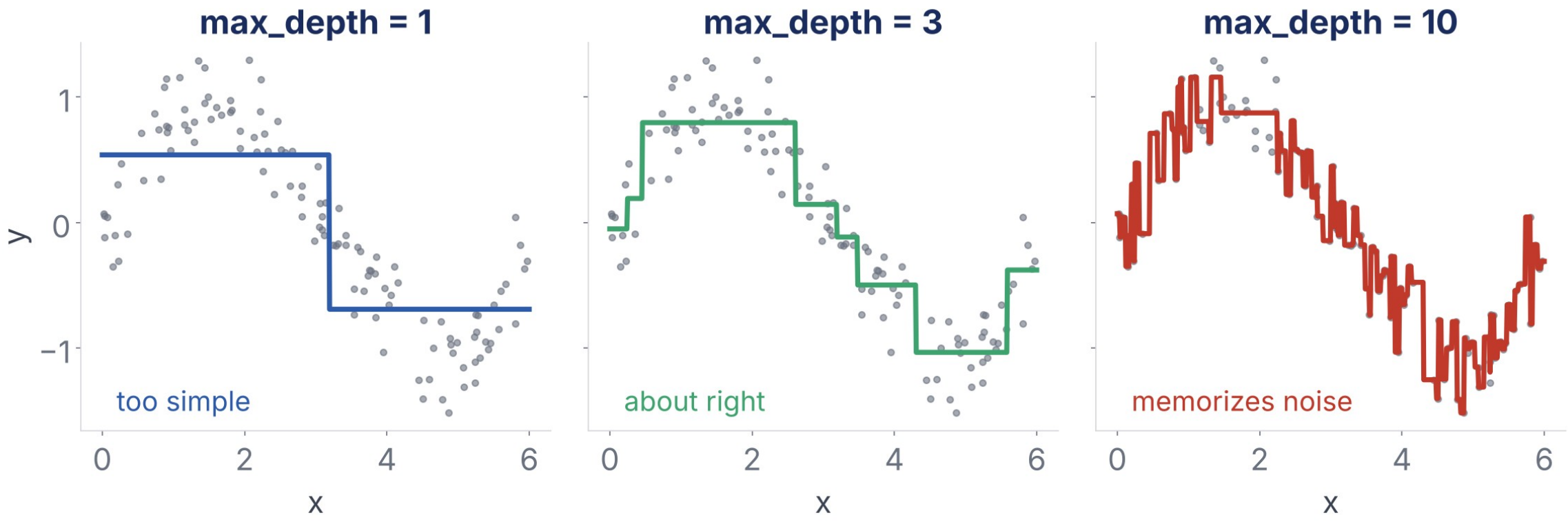
A good split creates purer children



weighted Gini after split = 0.32 → impurity drop = 0.18

Regression trees predict with a step function

Each leaf outputs the average y of its training points; noisy sine data, 120 points

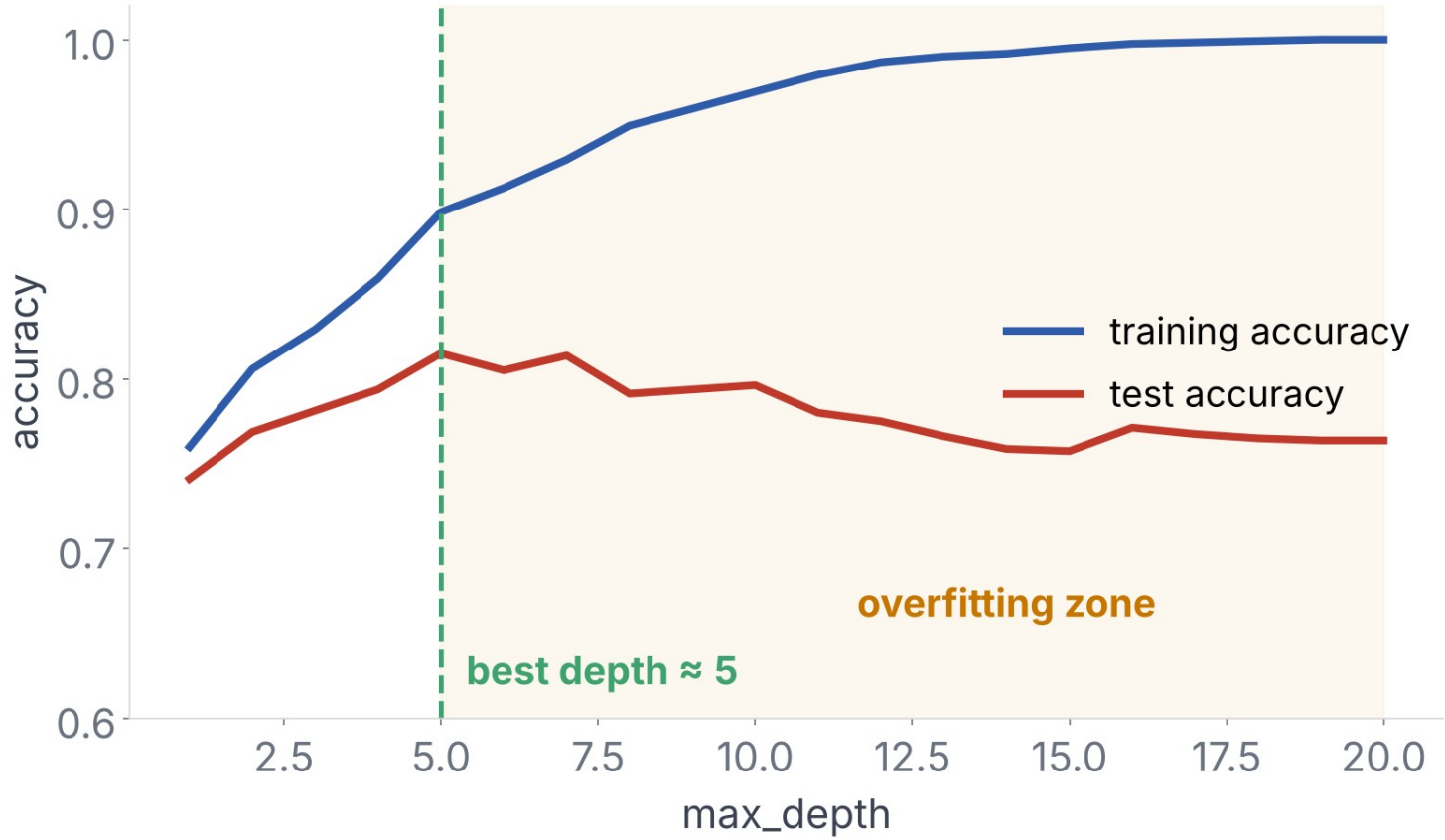


Deeper trees memorize the training data

Simulated data with 12% label noise; accuracy measured on held-out test rows

How to read it

- Training accuracy climbs to 100%.
- Test accuracy peaks early, then falls.
- The gap is overfitting: the tree learns noise.
- Fix: limit depth, prune, or average many trees.



A few settings control how far a tree grows

Names follow scikit-learn; other libraries use similar settings

Setting	What it controls	Effect when increased
max_depth	longest question path	more detail, more overfitting
min_samples_leaf	smallest allowed leaf	smoother, less overfitting
max_features	features tried per split	more accurate single splits
ccp_alpha	cost-complexity pruning	smaller tree, less overfitting

Rule of thumb: a single tree needs strong limits; a forest can use deep, unpruned trees

EPISODE

A three-question tree once beat doctors at triage

Cook County Hospital, Chicago, 1990s: Lee Goldman's chest-pain decision rule

ECG result + a few risk factors → send to cardiac unit, monitored bed, or home

Why it worked

- A few questions, asked the same way every time.
- Doctors weighed too much information inconsistently.
- The rule sorted heart-attack risk more reliably.

What it teaches

- Short trees are transparent and easy to audit.
- But one tree is fragile: new data may need a new tree.
- Ensembles gain accuracy, lose some transparency.

02

PART 2 · BAGGING & RANDOM FORESTS

Many Trees in Parallel

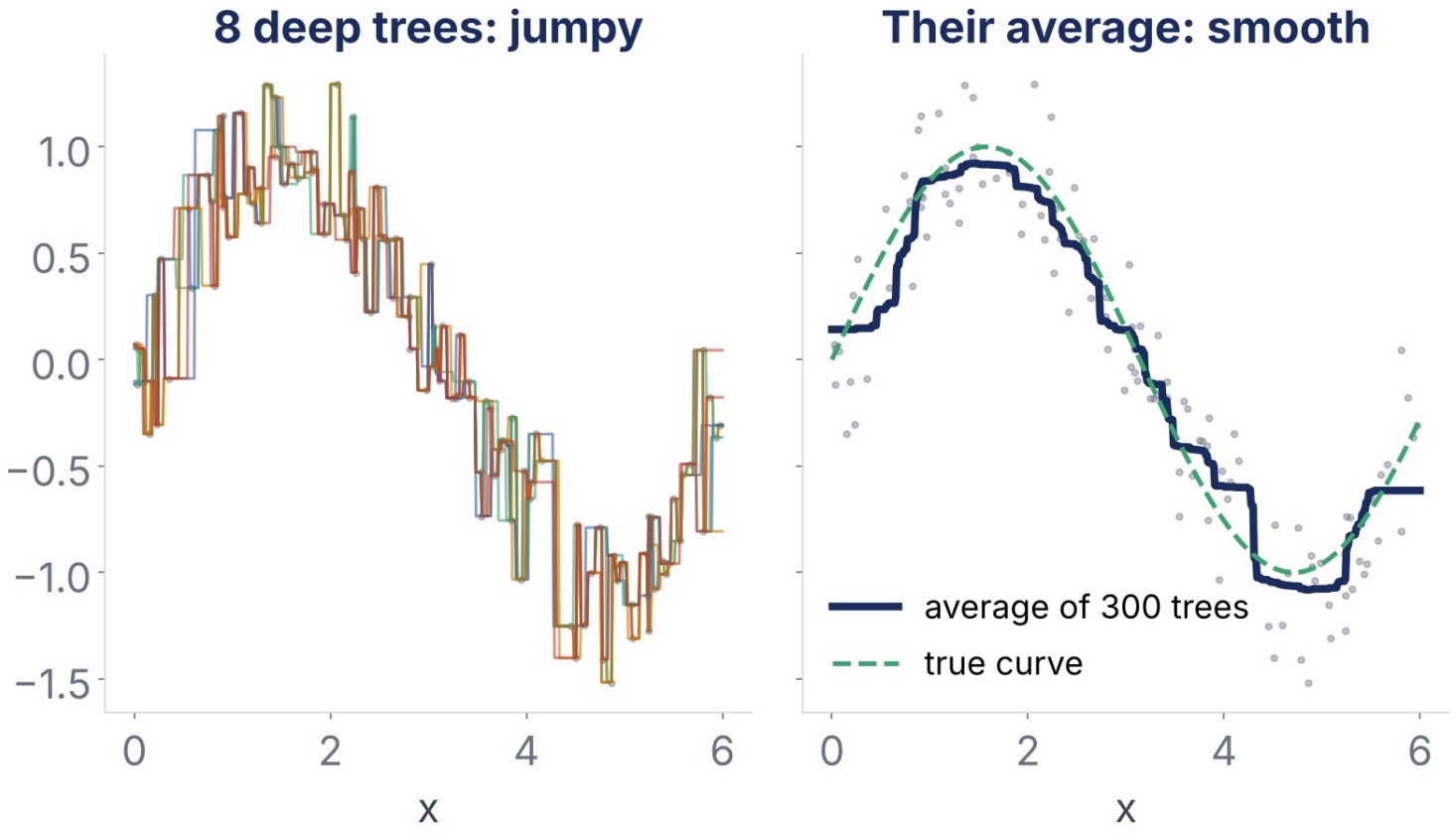
Bootstrap samples, random features and the power of averaging

Averaging many unstable trees cancels their noise

Left: deep trees on 8 bootstrap samples. Right: a 300-tree forest on the same data

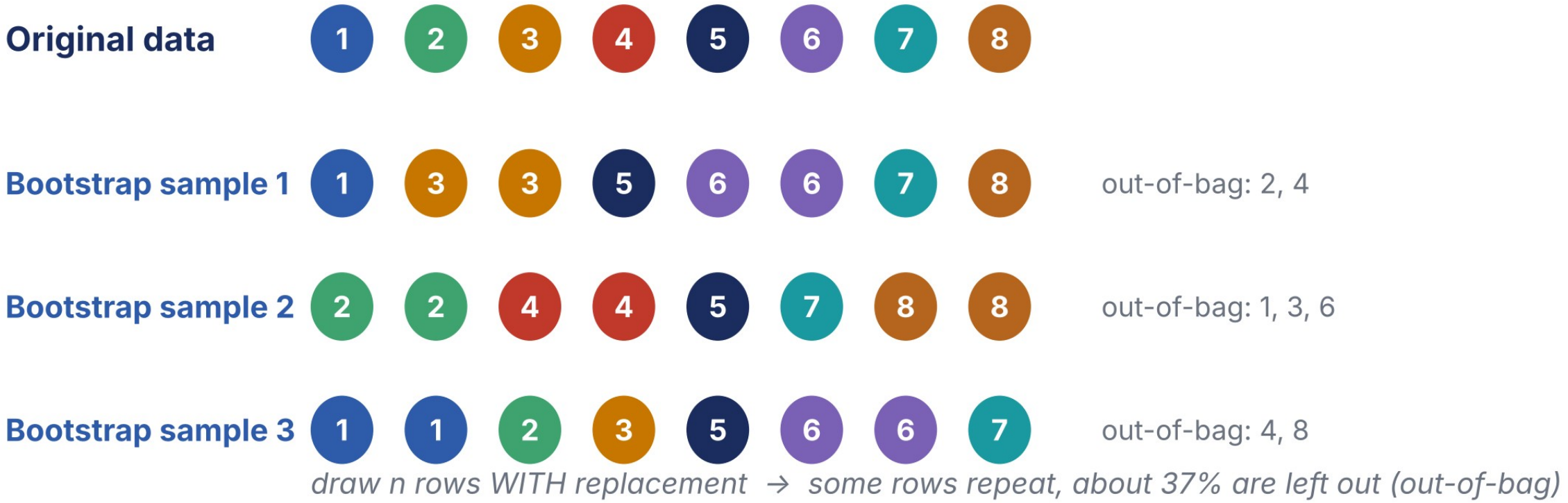
Key idea

- Each deep tree has low bias but high variance.
- Their errors point in different directions.
- The average keeps the signal and cancels much of the noise.



Bagging trains each tree on a bootstrap sample

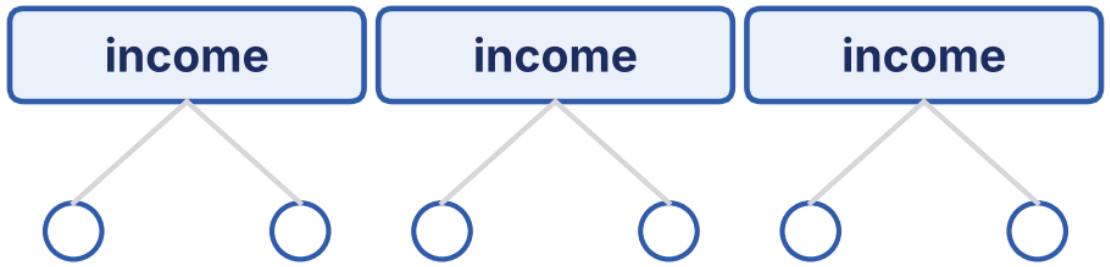
Bootstrap = draw n rows with replacement; rows not drawn are the tree's out-of-bag (OOB) rows



Random forests make the trees disagree on purpose

Each split may only look at a random few features, so trees grow differently

Bagging: every tree starts the same way



trees look alike
→ they make the same mistakes

averaging helps a little

Random forest: trees start differently



each split sees only a random few features
→ mistakes differ and cancel out

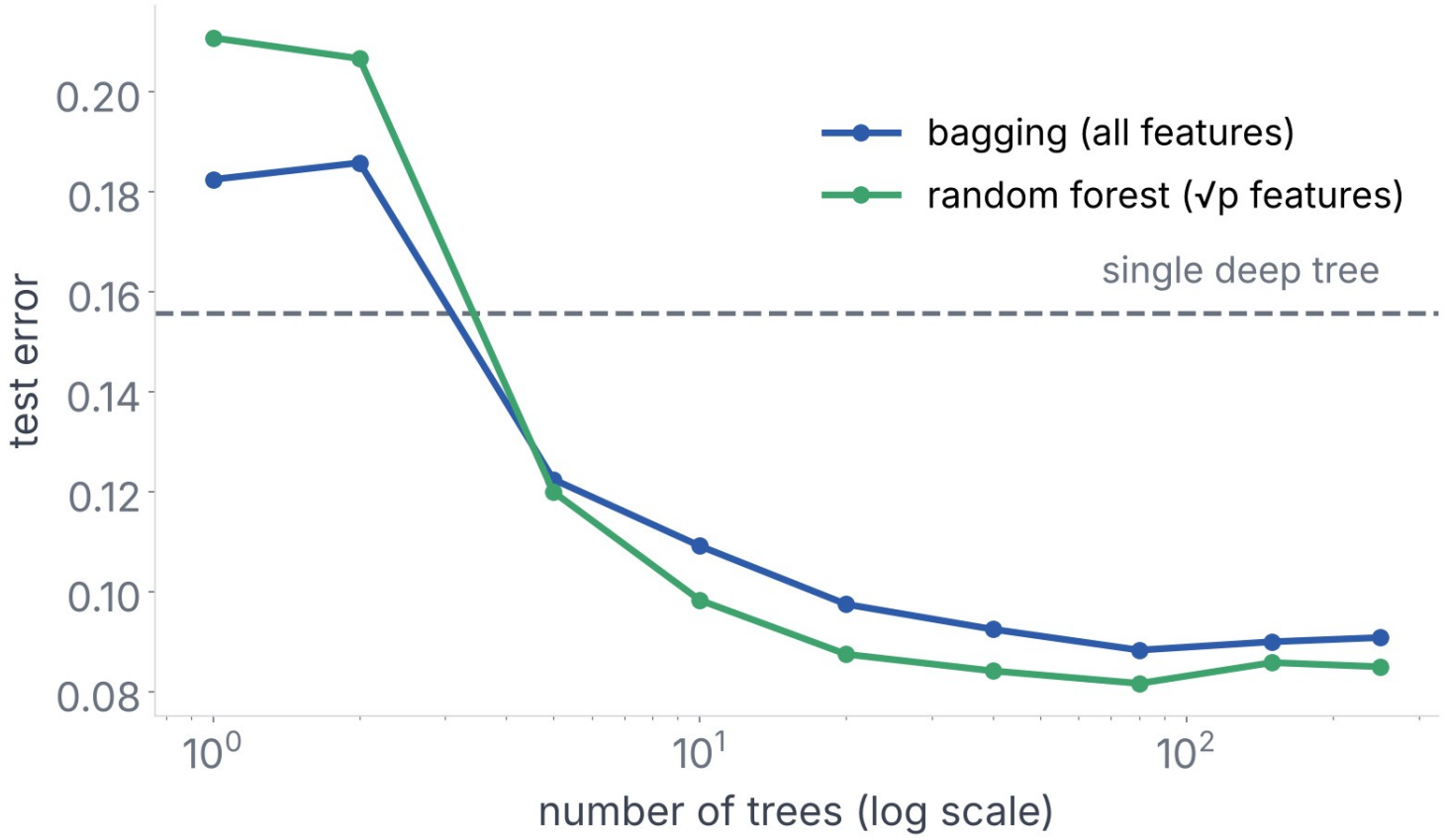
averaging helps a lot

Random features add a small extra gain over bagging

Simulated data, 30 features; test error against number of trees

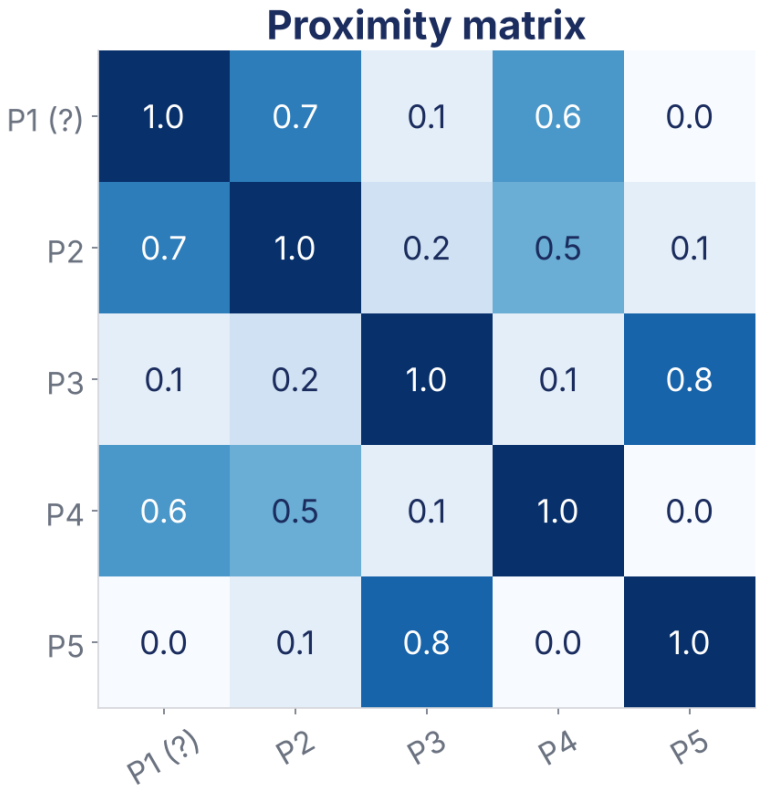
What we see

- One deep tree: 15.6% error.
- 250 bagged trees: 9.1%.
- 250-tree random forest: 8.5%.
- Gains flatten after about 100 trees.



Random forests fill blanks by asking similar rows

Breiman's idea: rows that often share a leaf are similar, so borrow their values



1 Initial guess — fill the blank with the median

2 Grow the forest — count how often two patients end in the same leaf

3 Weighted refill — new guess = proximity-weighted average of P2..P5 values

4 Repeat — re-grow and refill until the guesses stop changing

Step 1: start with a rough guess

Toy data with four patients; use only patients with the same label to make the first guess

Patient 4 has two blanks

#	Chest pain	Good blood flow	Blocked arteries	Weight	Heart disease
1	No	No	No	125	No
2	Yes	Yes	Yes	180	Yes
3	Yes	Yes	No	210	No
4	Yes	No	???	???	No

Rough first guess

#	Chest pain	Good blood flow	Blocked arteries	Weight	Heart disease
1	No	No	No	125	**No**
2	Yes	Yes	Yes	180	Yes
3	Yes	Yes	No	210	**No**
4	Yes	No	**No**	**167.5**	No

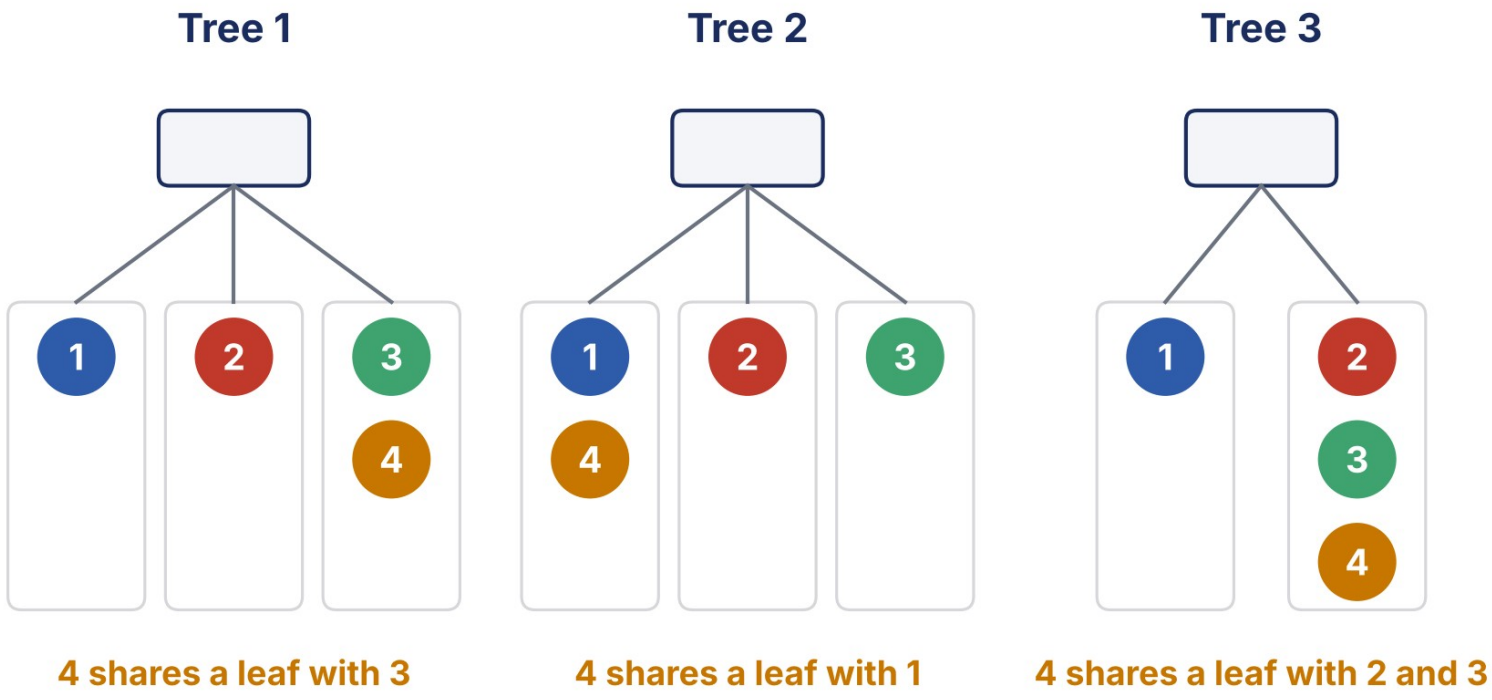
Look only at patients with the same label (no heart disease: #1 and #3)

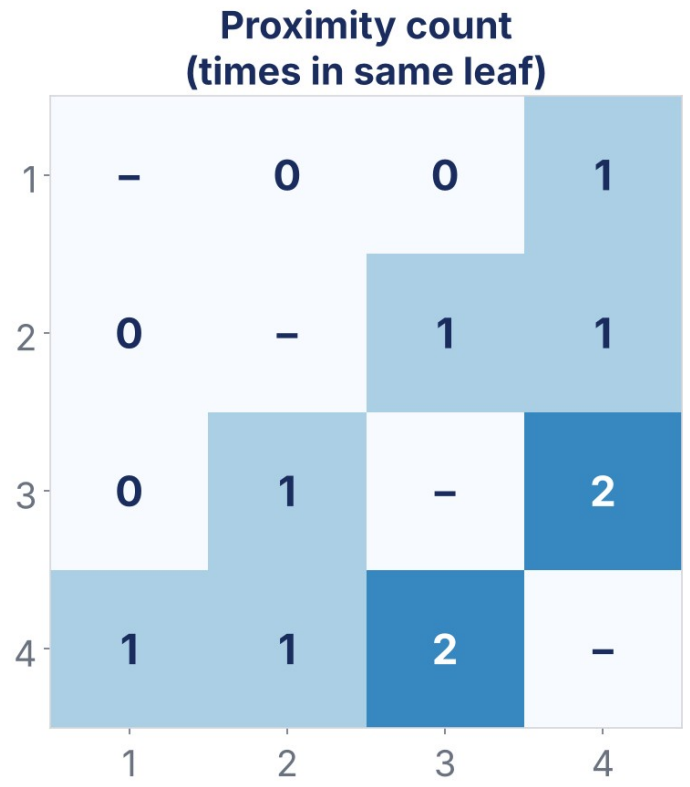
Blocked arteries → most common value: No

Weight → median of 125 and 210: 167.5

Step 2: grow a forest and count who shares a leaf

Proximity = how often two patients land in the same leaf; three trees shown, real forests use hundreds

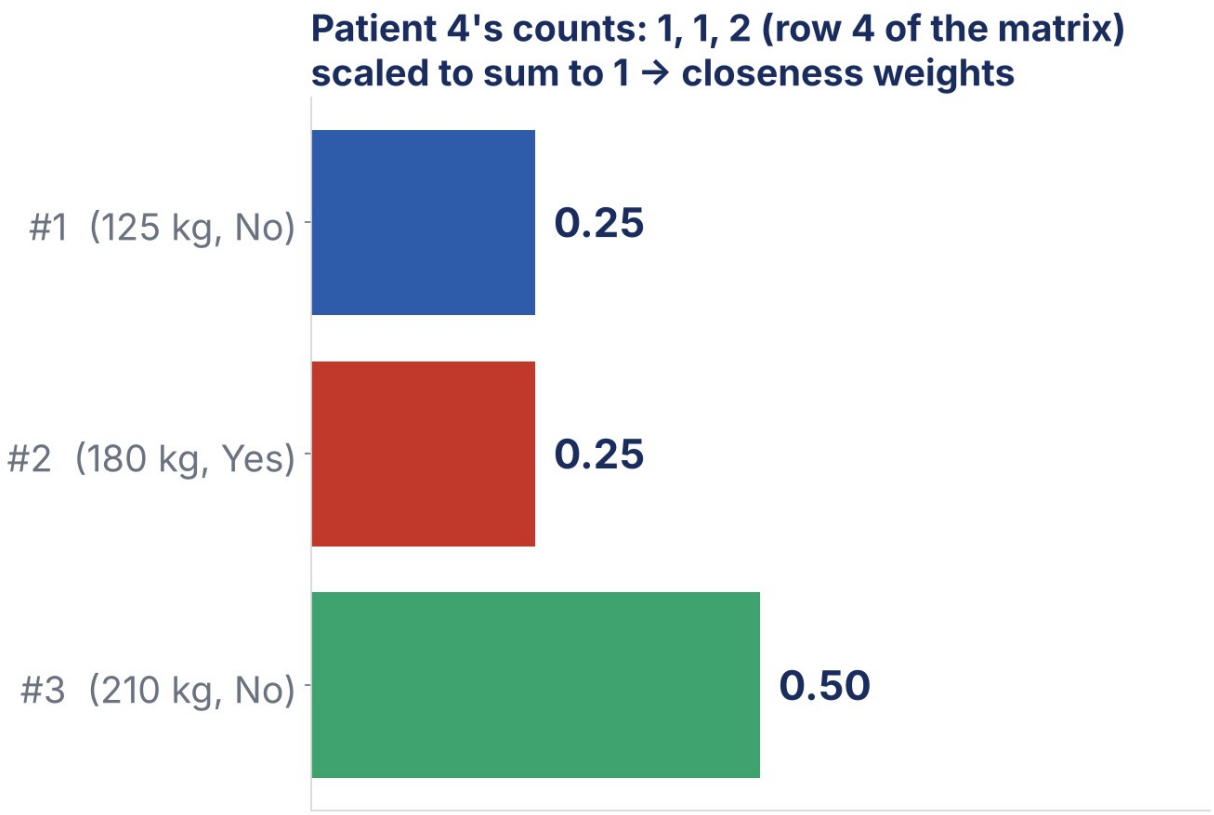




After each tree: +1 for every pair of patients that end in the same leaf

Step 3: refill each blank from its closest neighbors

Numbers use a weighted average; categories use a weighted vote



Weight (a number): weighted average

0.25 × 125 + 0.25 × 180 + 0.50 × 210

= 181.3 kg (first guess was 167.5)

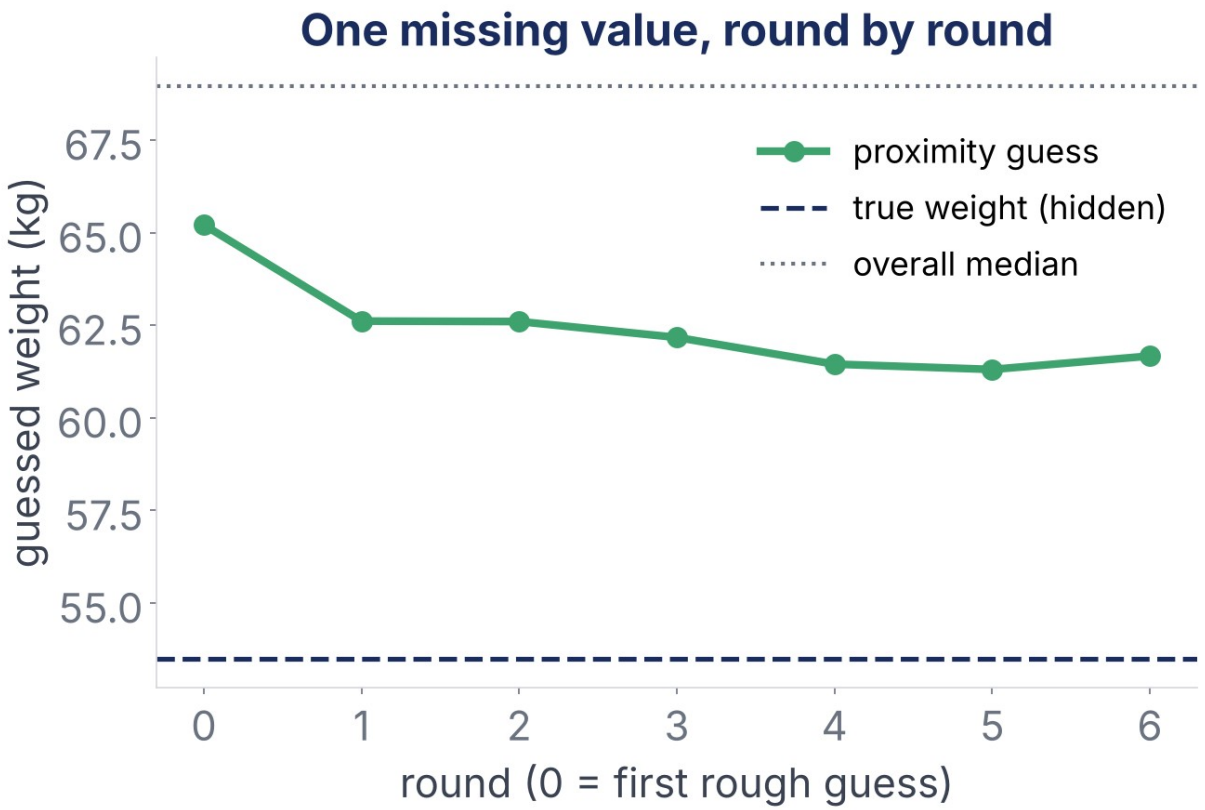
Blocked arteries (a category): weighted vote

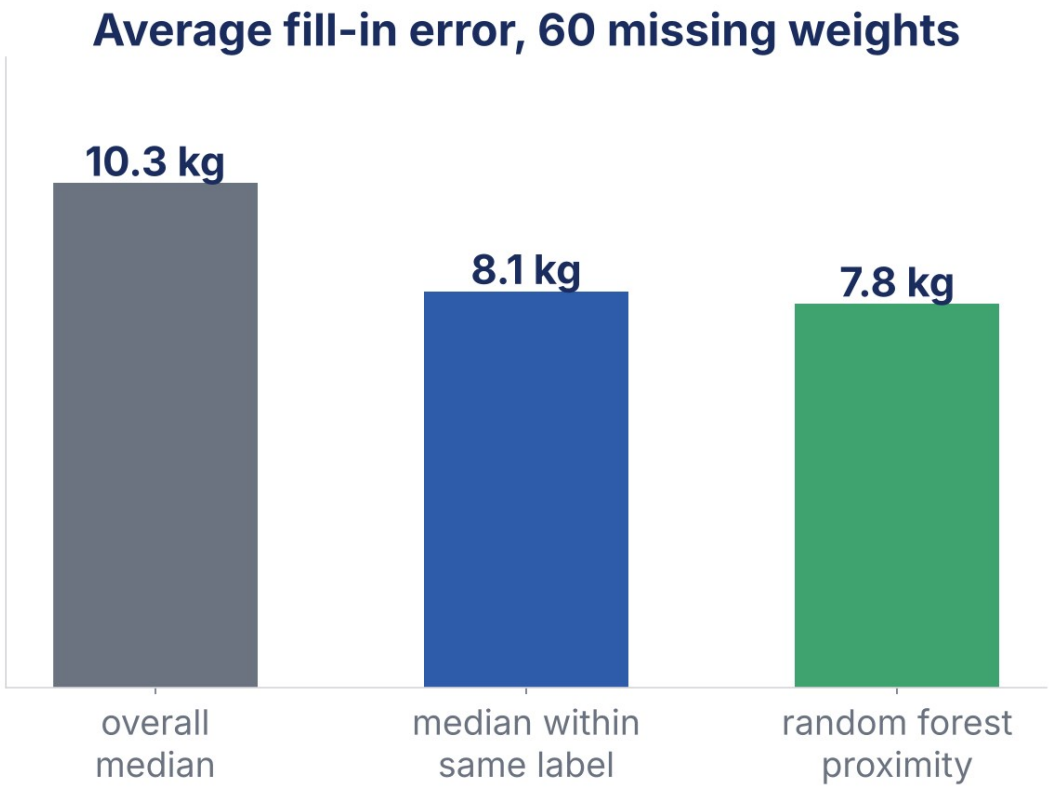
Yes 0.25 (patient 2) → fill with No

No 0.75 (patients 1, 3)

Step 4: repeat until the guesses settle

Simulated data: 300 patients, 20% of weights hidden, forest re-grown each round





A new row without a label is handled with copies

Try each possible label, fill the blank for it, and keep the copy the forest labels correctly most often

A new patient arrives: we do not know the label, and one value is missing

new	Yes	No	???	168	?
copy A	Yes	No	No	168	Yes
copy B	Yes	No	No	168	No

make one copy per possible label

fill each blank with the rough guess for that label

Run both copies down the forest

copy A: share of trees that vote Yes (example)



copy B: share of trees that vote No (example)



Copy A is labeled correctly more often → keep it

Proximity filling: strengths and limits

When to use it, and what to watch out for

Similar rows lend their values: the forest decides what “similar” means

Strengths

- Works for numbers and categories.
- Uses all other features, not just one column.
- No distance formula or scaling needed.

Limits

- Slow: a new forest every round.
- Gains over a same-label median can be small.
- Tools: rfImpute in R; MissForest is a close cousin.

03

PART 3 · BOOSTING

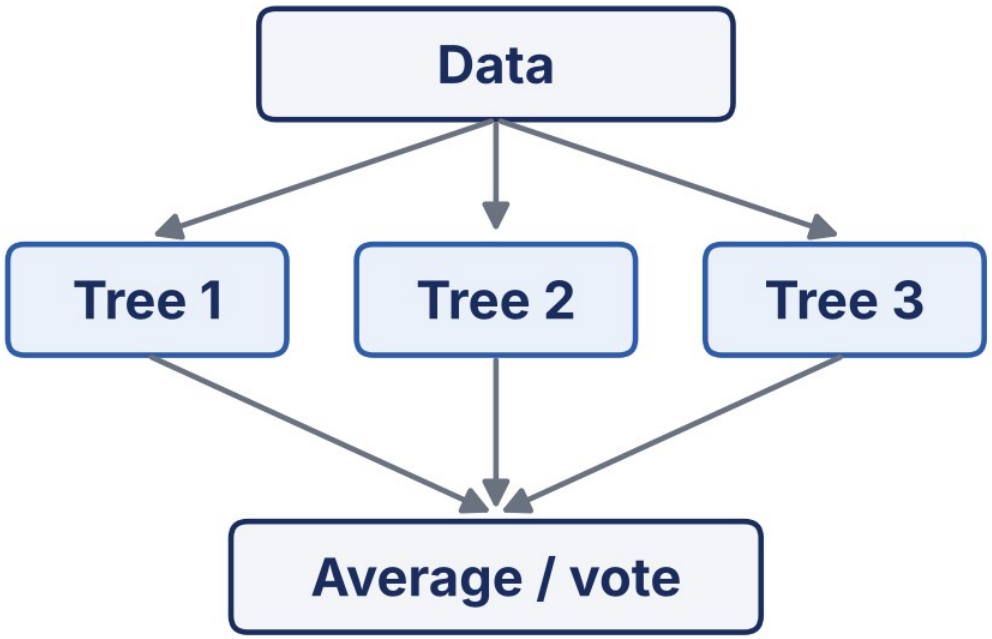
Many Small Trees in Sequence

AdaBoost and gradient boosting: each tree learns from earlier mistakes

Bagging averages trees; boosting chains them

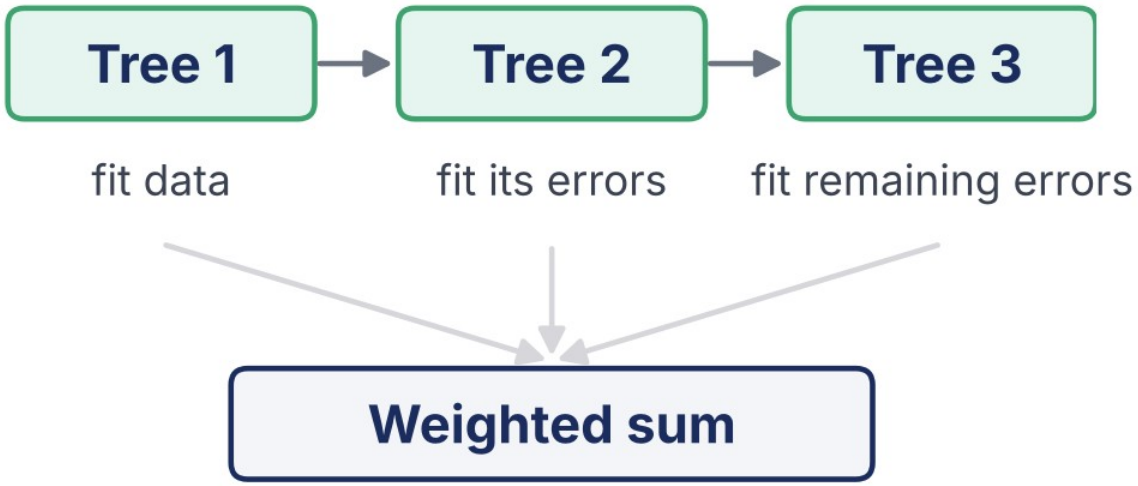
Bagging attacks variance, boosting attacks bias

BAGGING / RANDOM FOREST: in parallel



independent deep trees → lower variance

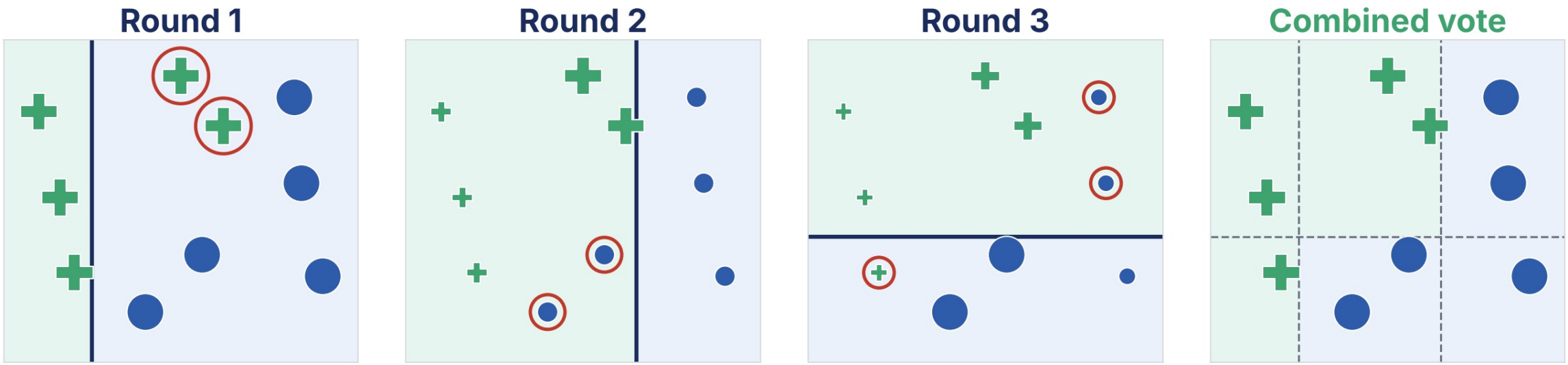
BOOSTING: in sequence



small trees fix each other → lower bias

AdaBoost reweights the points it got wrong

The classic toy example: each round draws one straight cut; red rings mark mistakes



one cut, 2 mistakes

mistakes from round 1 now look bigger

the next cut focuses on the big points

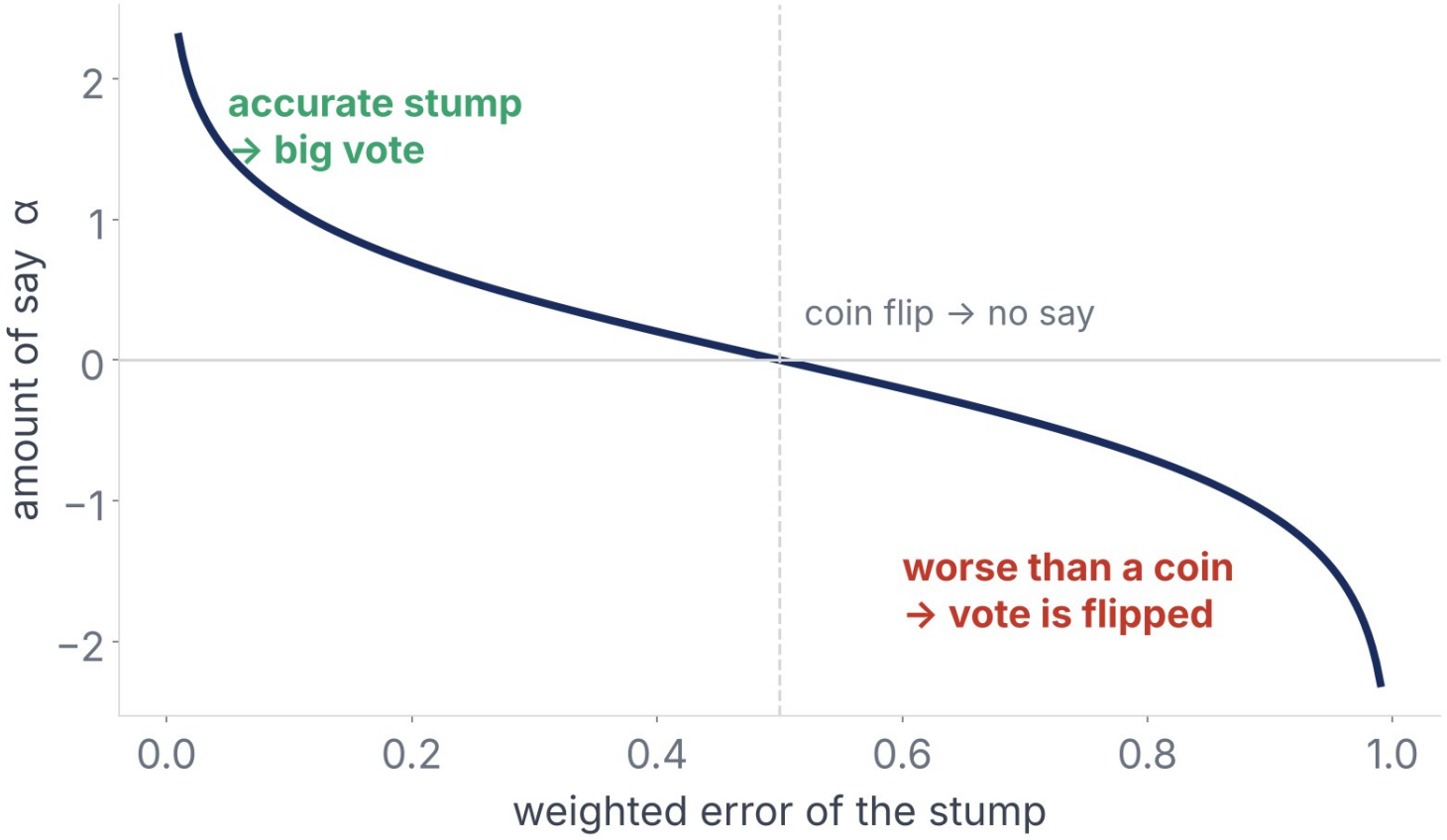
three weak cuts together classify every point

Accurate cuts receive a larger vote

Each cut votes; better cuts count more in the final decision

Reading the curve

- Few mistakes → loud vote.
- Coin-flip cut → no vote.
- Worse than a coin → its vote is flipped.



AdaBoost put face detection into every camera

Viola and Jones (2001) chained boosted stumps into a fast cascade

Thousands of simple light-dark patterns, each one a weak stump, combined by AdaBoost

Why boosting fit

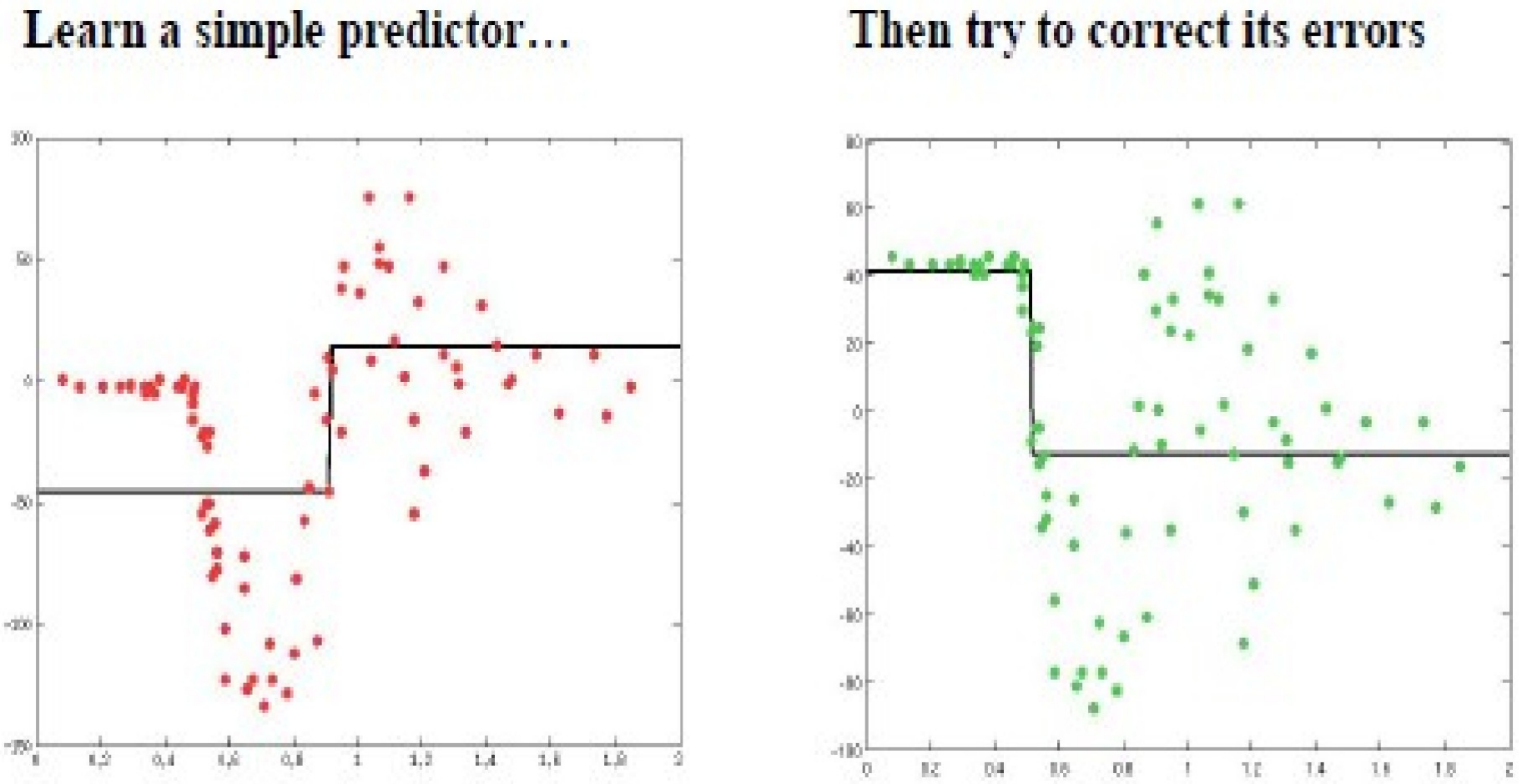
- Each pattern alone is barely better than guessing.
- Boosting picks the useful few and weights them.
- Cheap stages reject non-faces early, so it runs in real time.

Recognition

- Freund and Schapire received the 2003 Gödel Prize for AdaBoost.
- The idea of turning weak learners into a strong one led to gradient boosting.

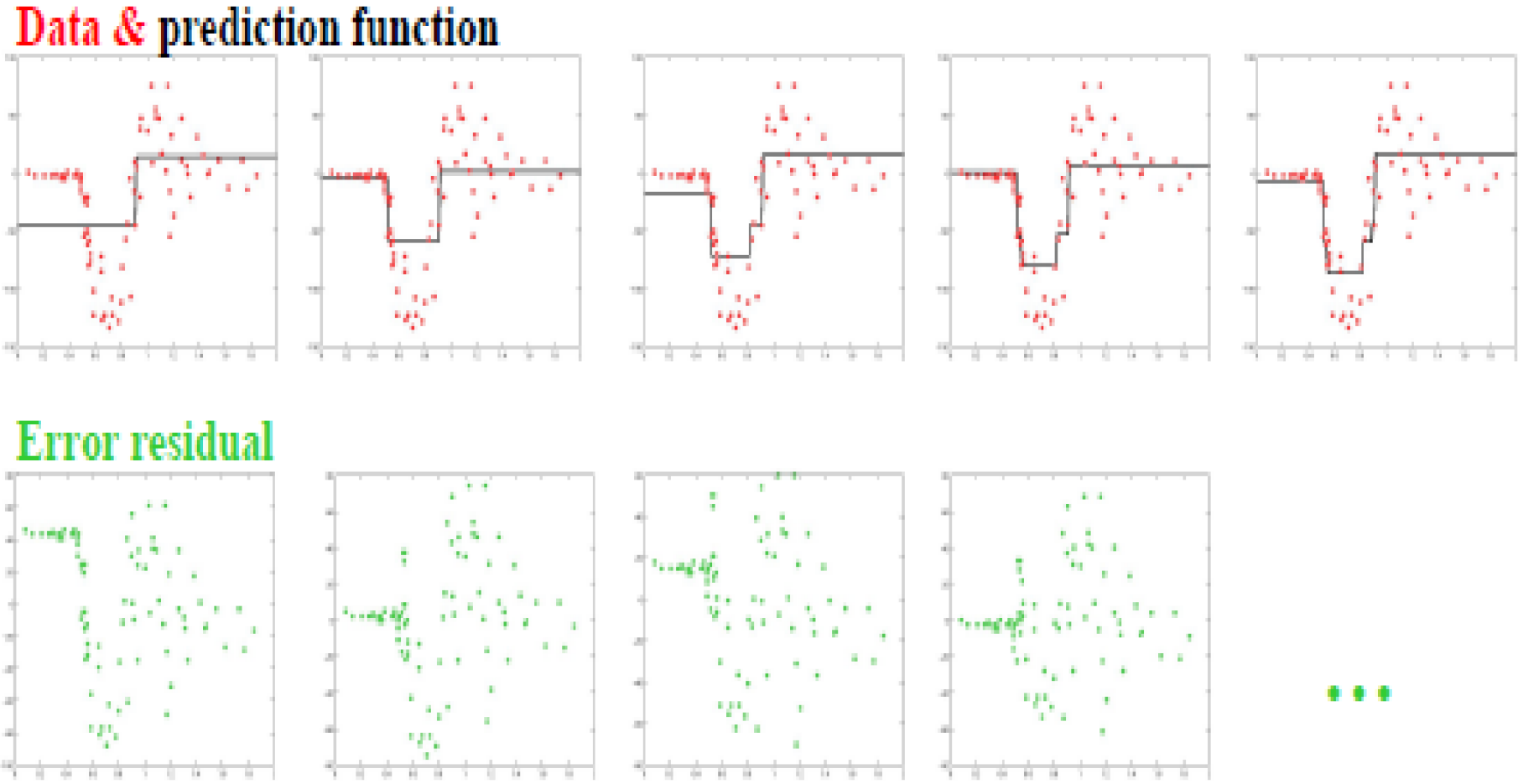
Boosting learns a simple model, then fixes its errors

Left: a first small tree. Right: a second tree trained only on what the first one got wrong



Each round fits the errors left by the round before

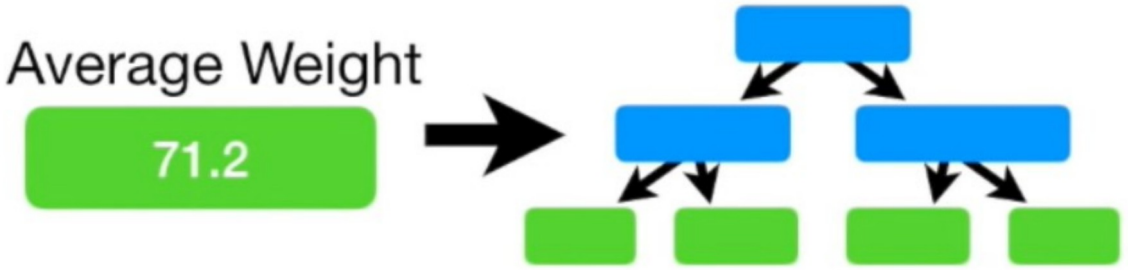
Top: data and prediction after each round. Bottom: the leftover errors (residuals) the next tree learns



Step 1: start by predicting the average weight

Six people with height, favorite color and gender; the first guess for everyone is the average, 71.2 kg

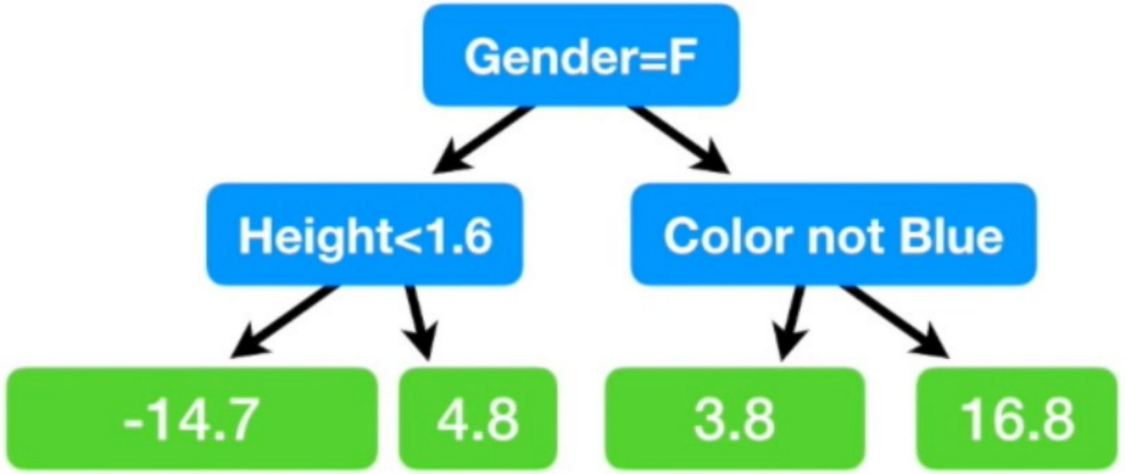
Height (m)	Favorite Color	Gender	Weight (kg)
1.6	Blue	Male	88
1.6	Green	Female	76
1.5	Blue	Female	56
1.8	Red	Male	73
1.5	Green	Male	77
1.4	Blue	Female	57



Step 2: a small tree learns the residuals

Residual = observed weight minus 71.2; when two residuals share a leaf, the leaf keeps their average

Height (m)	Favorite Color	Gender	Weight (kg)	Residual
1.6	Blue	Male	88	16.8
1.6	Green	Female	76	4.8
1.5	Blue	Female	56	-15.2
1.8	Red	Male	73	1.8
1.5	Green	Male	77	5.8
1.4	Blue	Female	57	-14.2



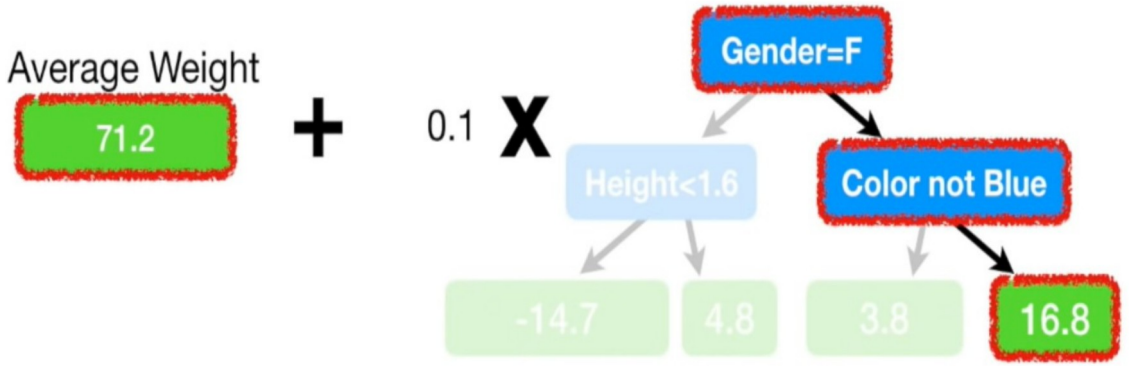
Step 3: a full step memorizes, a small step generalizes

Full step: person 1 is predicted at exactly 88 kg. Learning rate 0.1: only 72.9 kg

Predicted Weight = 71.2 + 16.8 = 88

Height (m)	Favorite Color	Gender	Weight (kg)
1.6	Blue	Male	88

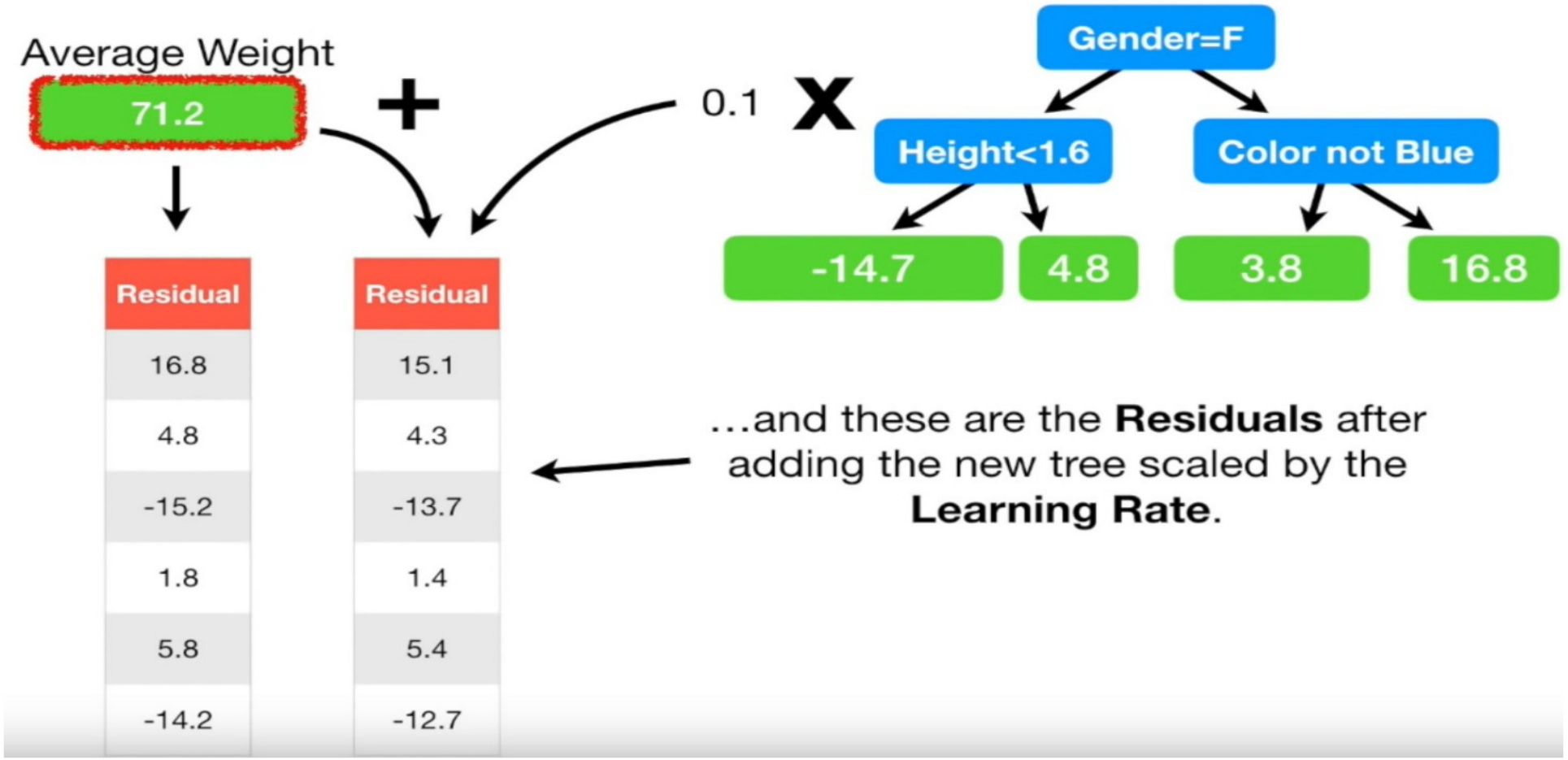
…which is the same as the **Observed Weight**.



Now the **Predicted Weight** = 71.2 + (0.1 x 16.8) = 72.9

Step 4: the residuals shrink after one small step

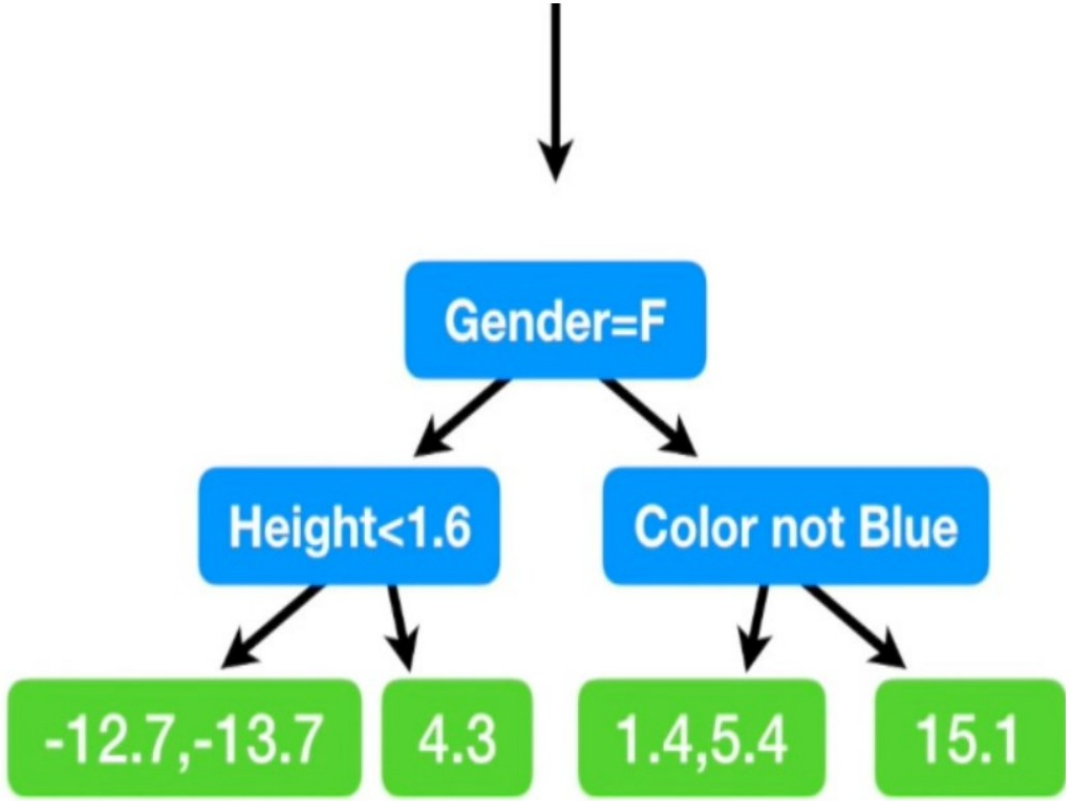
Left column: first residuals (from the average). Right column: residuals after adding 0.1 × tree 1



Step 5: a second tree learns the new residuals

Same features, new targets; leaves with two rows again keep the average

Height (m)	Favorite Color	Gender	Weight (kg)	Residual
1.6	Blue	Male	88	15.1
1.6	Green	Female	76	4.3
1.5	Blue	Female	56	-13.7
1.8	Red	Male	73	1.4
1.5	Green	Male	77	5.4
1.4	Blue	Female	57	-12.7



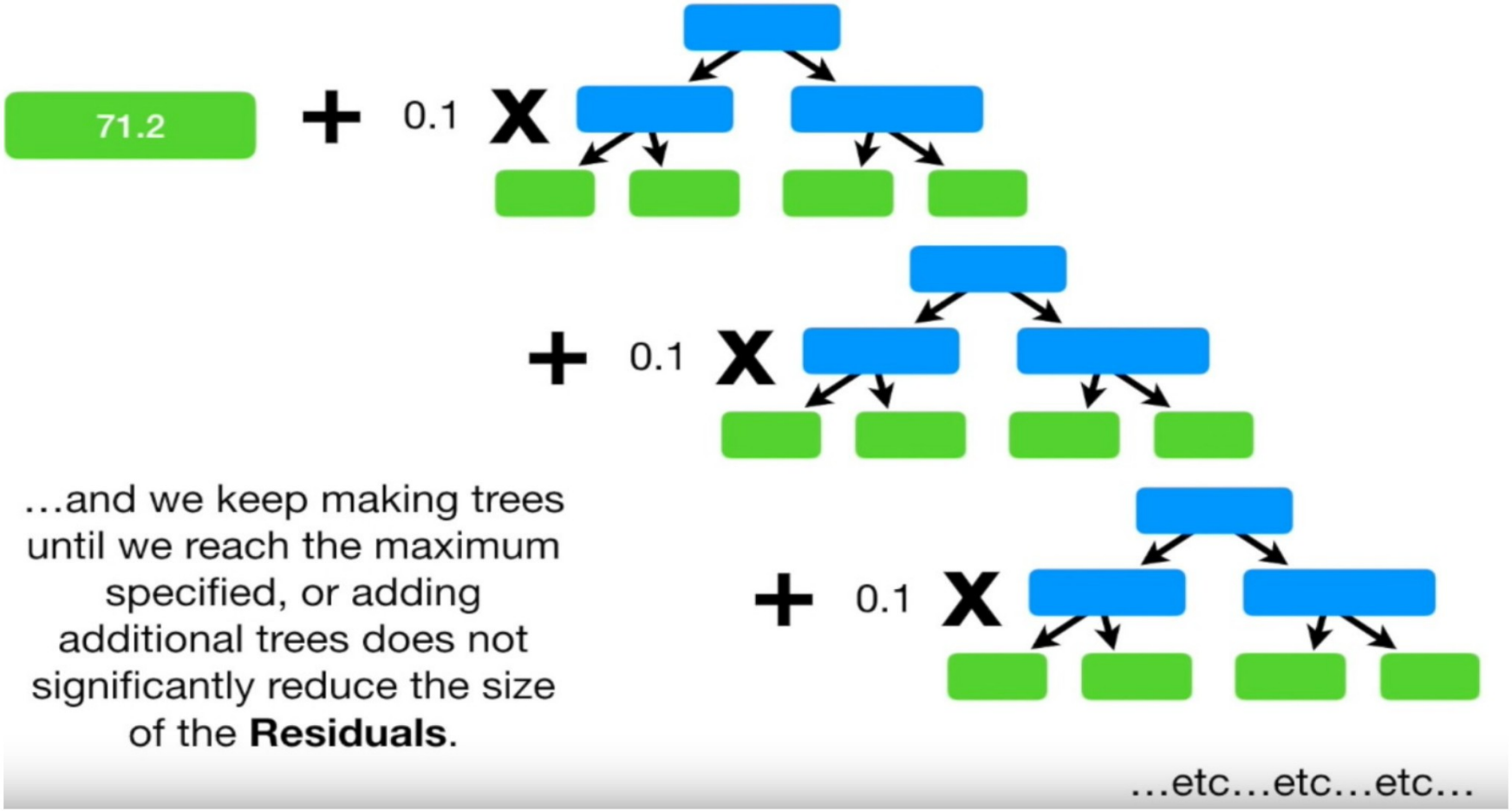
Step 6: two small trees move the guess closer

Person 1: 71.2 + 0.1 × 16.8 + 0.1 × 15.1 = 74.4 kg; the true weight is 88 kg



Step 7: keep adding small corrections

Stop at a set number of trees, or when extra trees no longer shrink the residuals

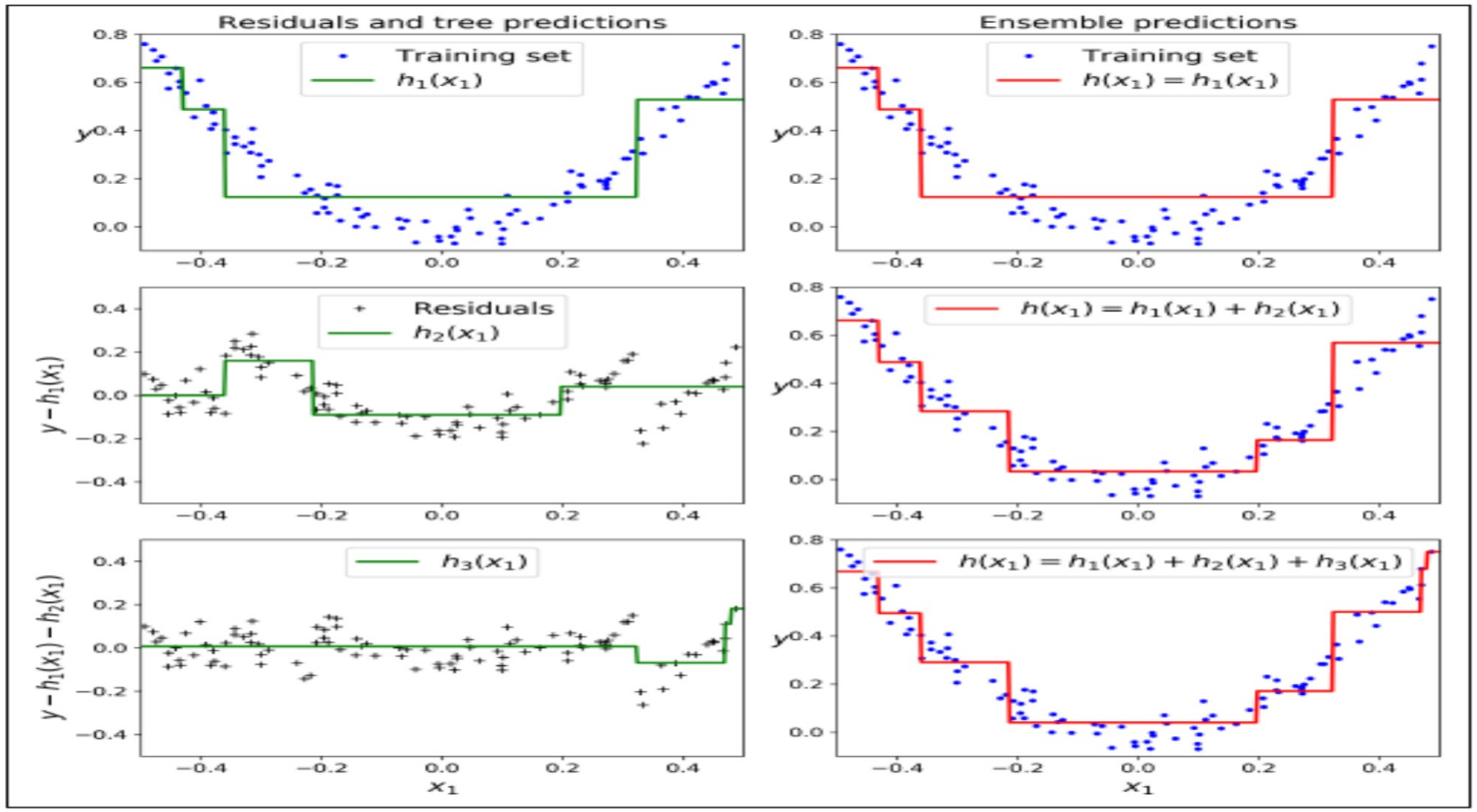


The ensemble is the sum of all the small trees

Three depth-2 trees in scikit-learn: left, each tree's fit; right, the running total

How to read it

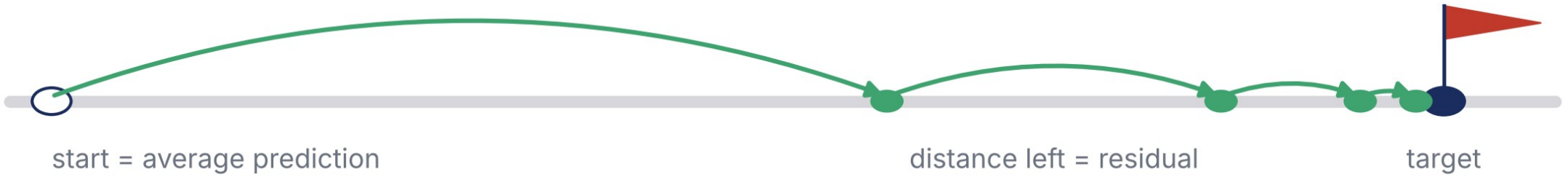
- Row 1: tree 1 fits the data.
- Row 2: tree 2 fits tree 1's errors.
- Row 3: tree 3 fits what is still left.
- Right column: the prediction gets closer each row.



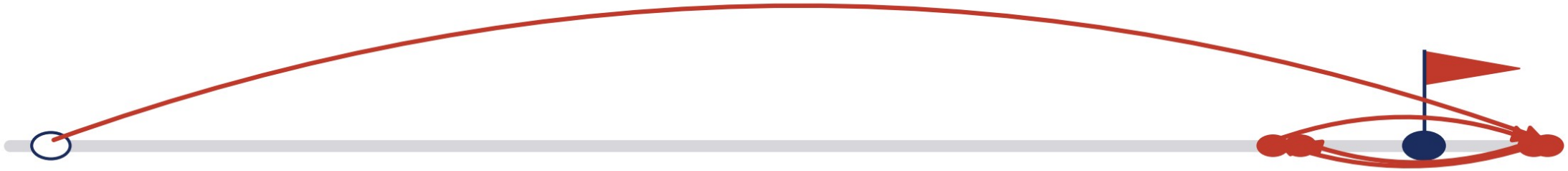
Boosting works like putting a golf ball

Each tree is one shot toward the target; the learning rate sets how hard you hit

Small, careful shots: each tree covers part of the remaining distance



Full-power shots: the ball keeps overshooting the hole

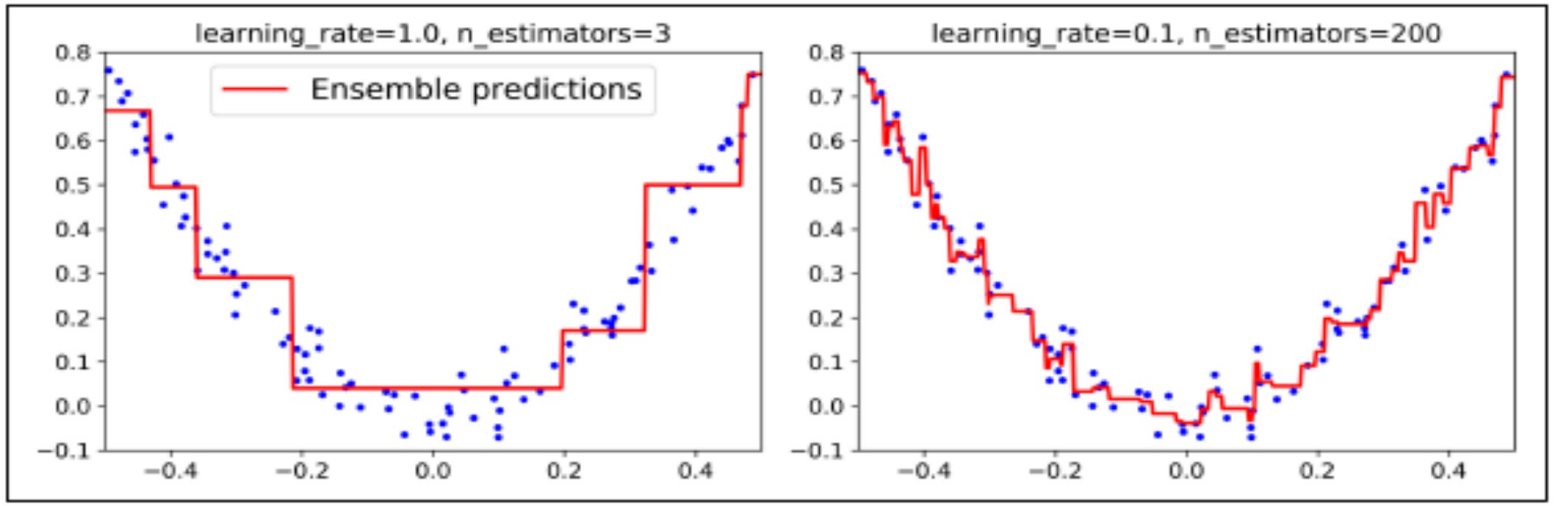


Small steps need more trees but fit better

Left: learning rate 1.0 with 3 trees. Right: learning rate 0.1 with 200 trees

Practical rule

- Big steps: few trees, rough fit.
- Small steps: many trees, smooth fit.
- Use a small rate and stop when validation error stops improving.



04

PART 4 · MODERN GBDT

XGBoost, LightGBM and CatBoost

Same boosting idea, different engineering and regularization

XGBoost keeps the idea and upgrades the engine

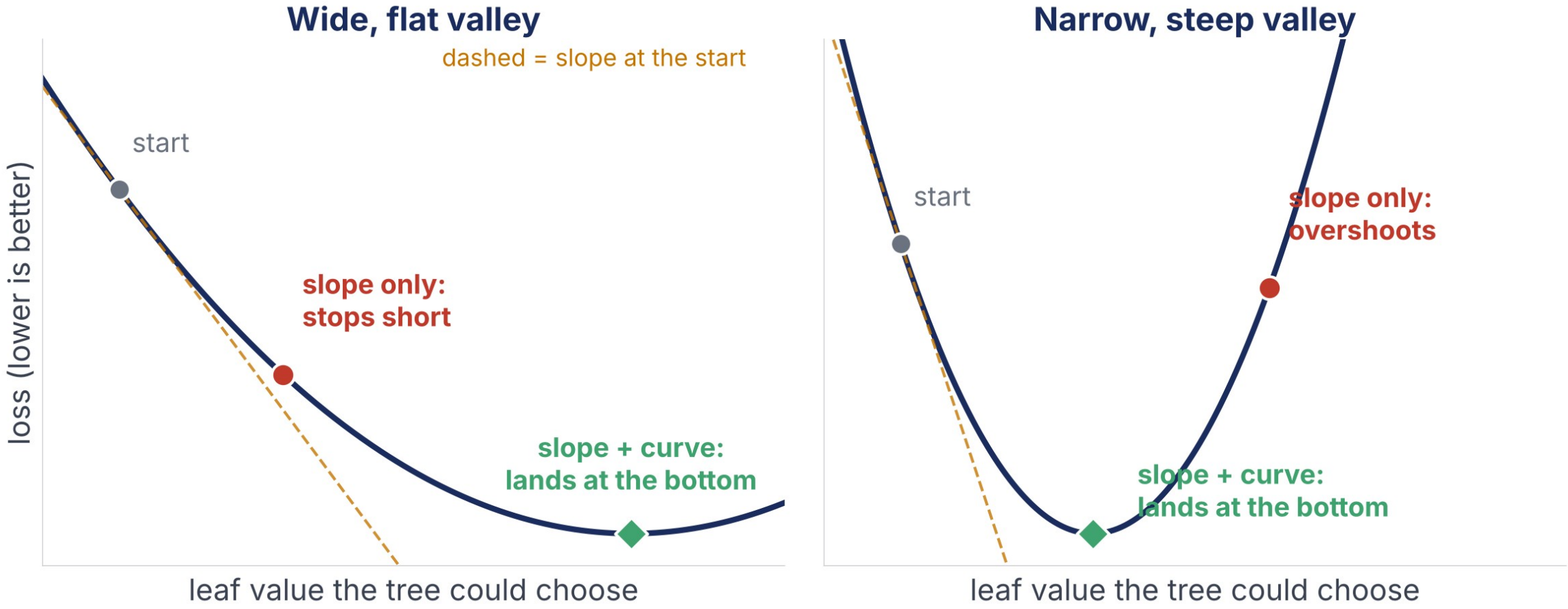
Both add small trees that fix earlier errors; the differences are in how

Question	Classic gradient boosting	XGBoost
How far does each tree step?	uses the slope only	uses slope and curvature
How is tree size kept small?	depth limit only	penalty inside every split
What about missing values?	fill them in first	learns where to send them
Which cuts are tried?	every gap between values	bin edges only
How many CPU cores?	one	all of them

The next slides show each difference as a picture

XGBoost feels the curve, not just the slope

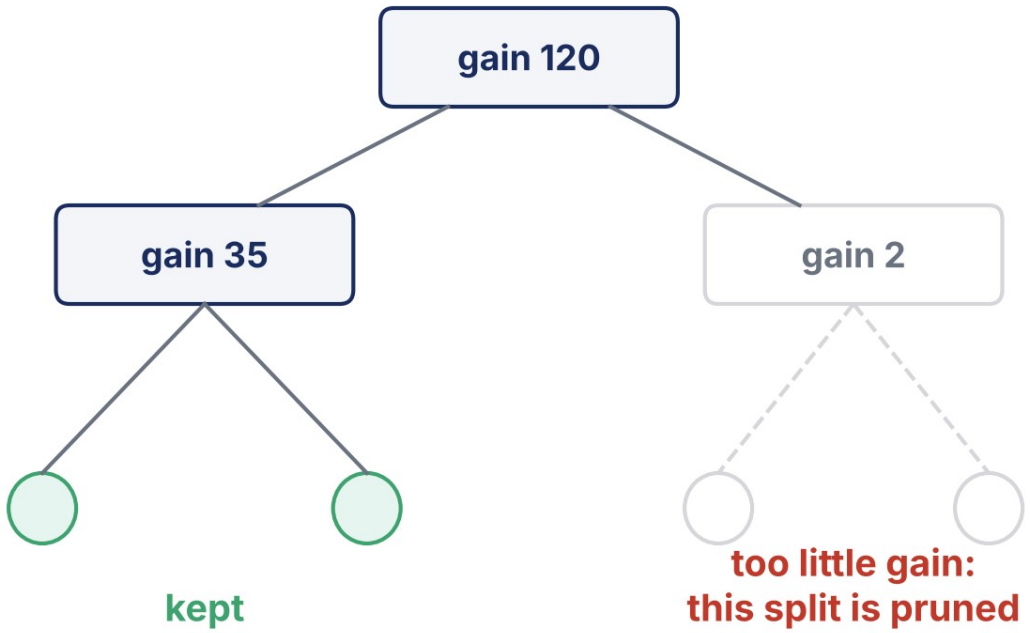
Like walking downhill in fog: the slope gives the direction, the curve tells how far the bottom is



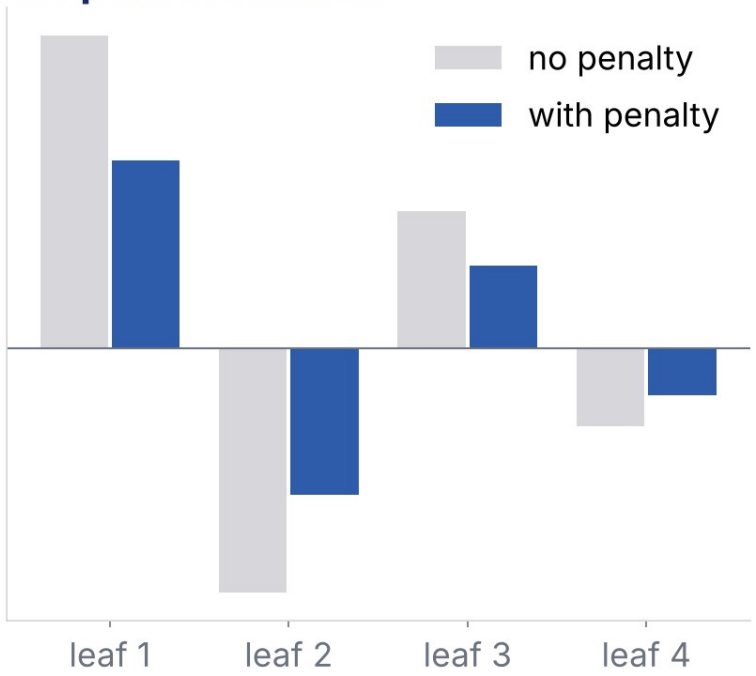
XGBoost adds penalties that keep trees small

Weak splits are pruned and leaf outputs are shrunk, so each tree makes a modest step

Penalty 1: a split must earn its place

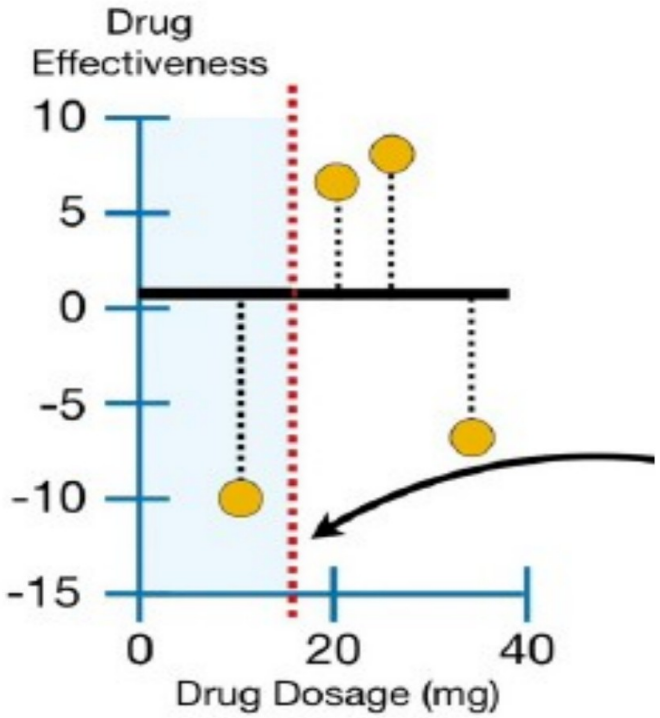


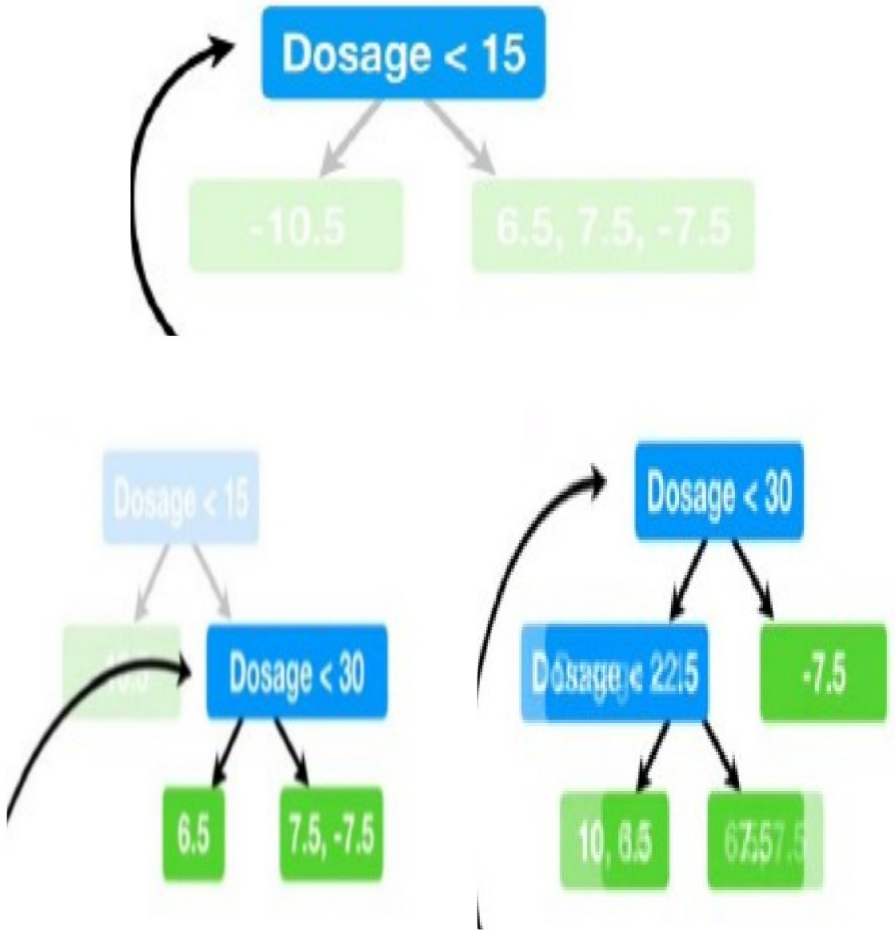
Penalty 2: leaf values are pulled toward 0



XGBoost keeps the cut with the best gain

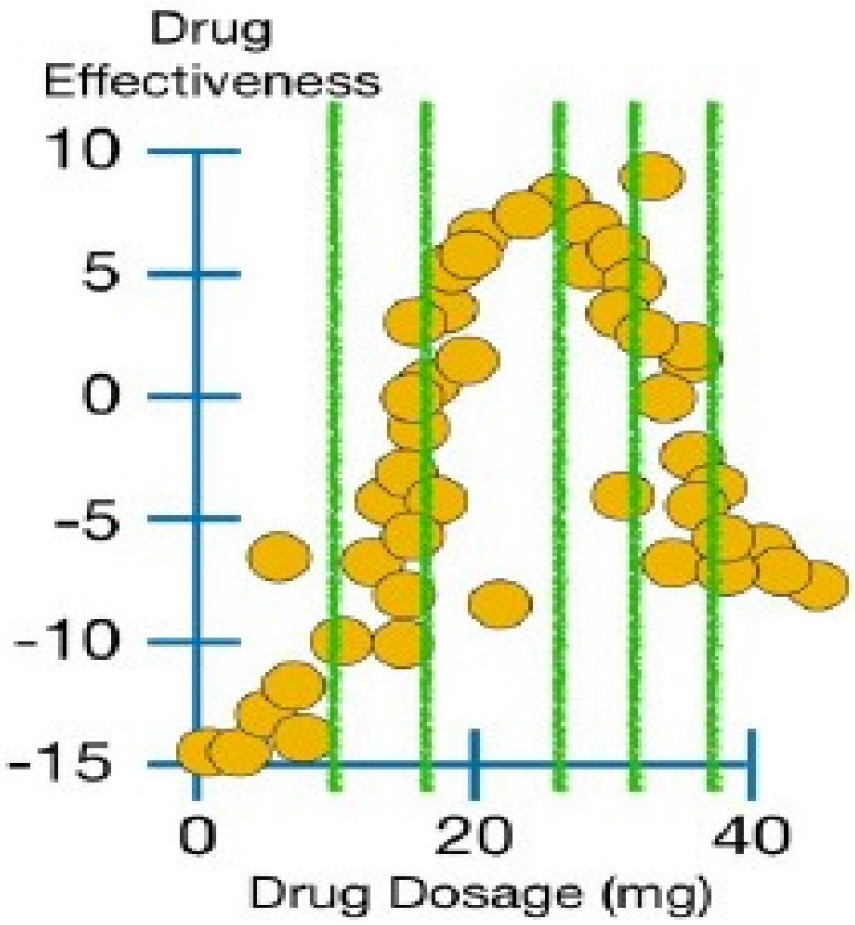
Drug-dosage example: every candidate cut is scored; the best one becomes the split

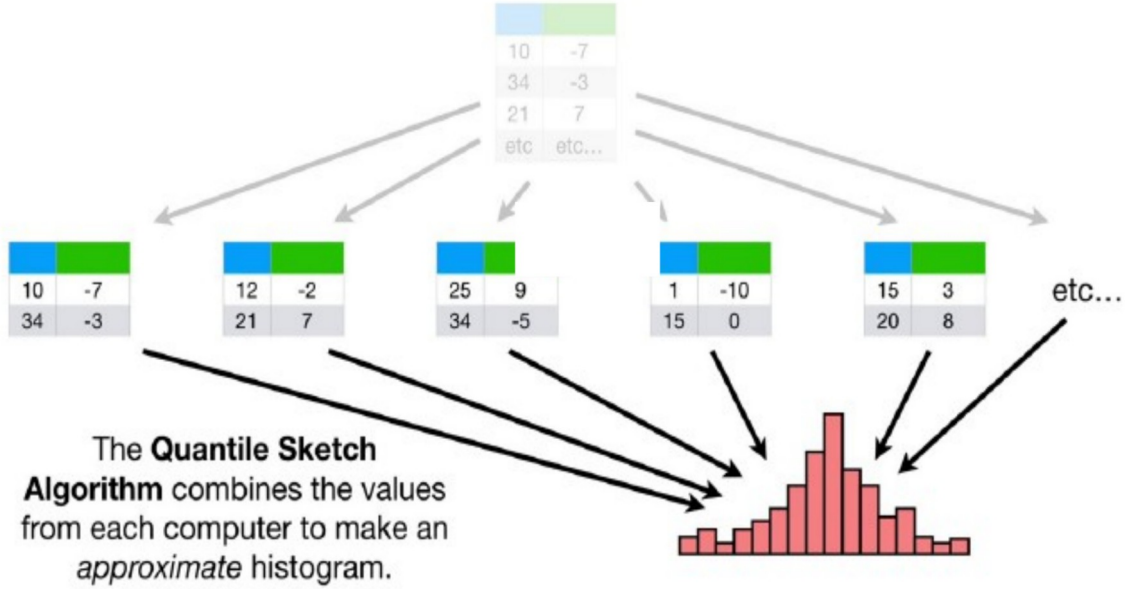




On big data, XGBoost tests only quantile cut points

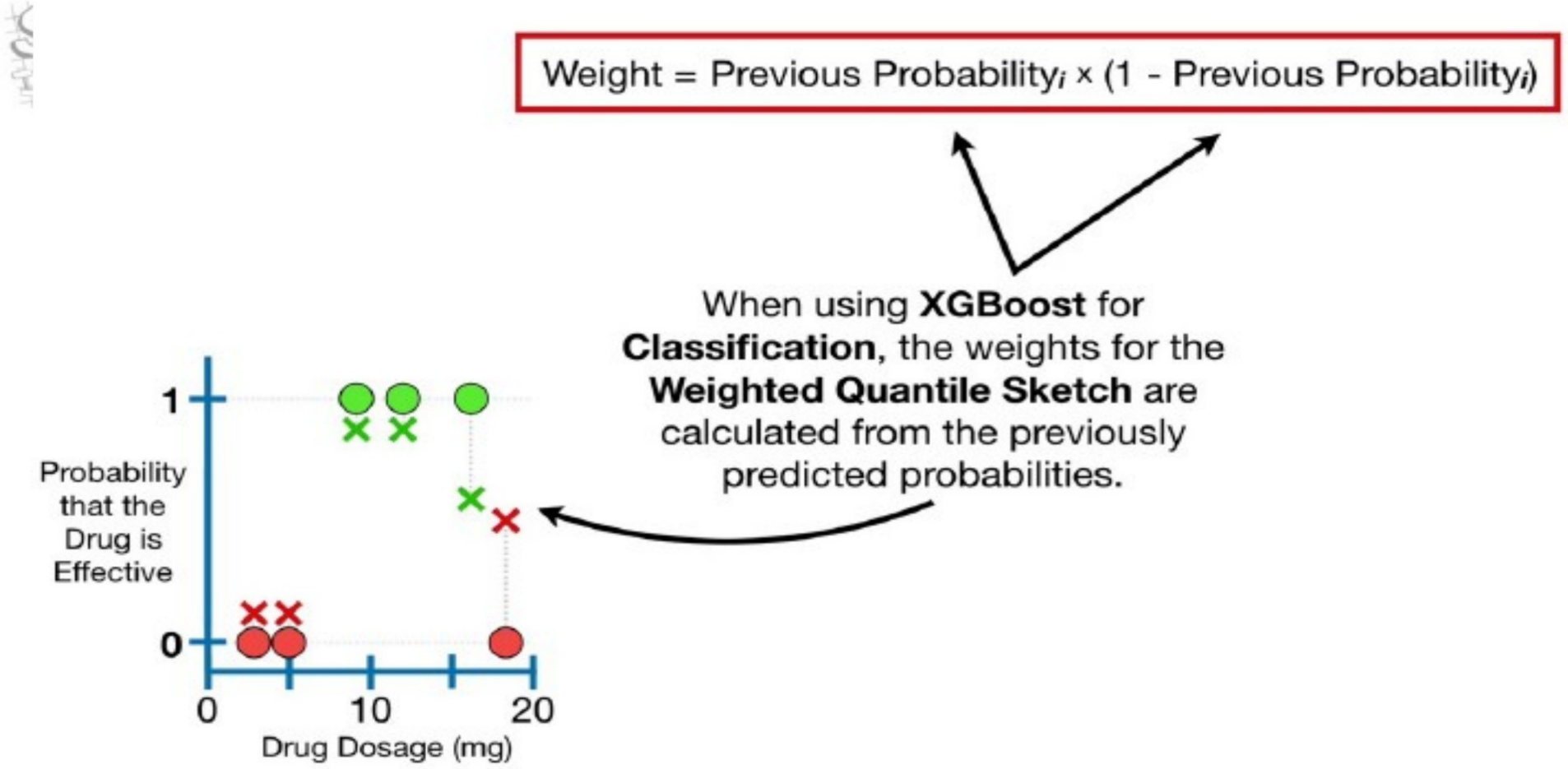
Left: cuts at quantiles only. Right: summaries from many machines merge into one histogram





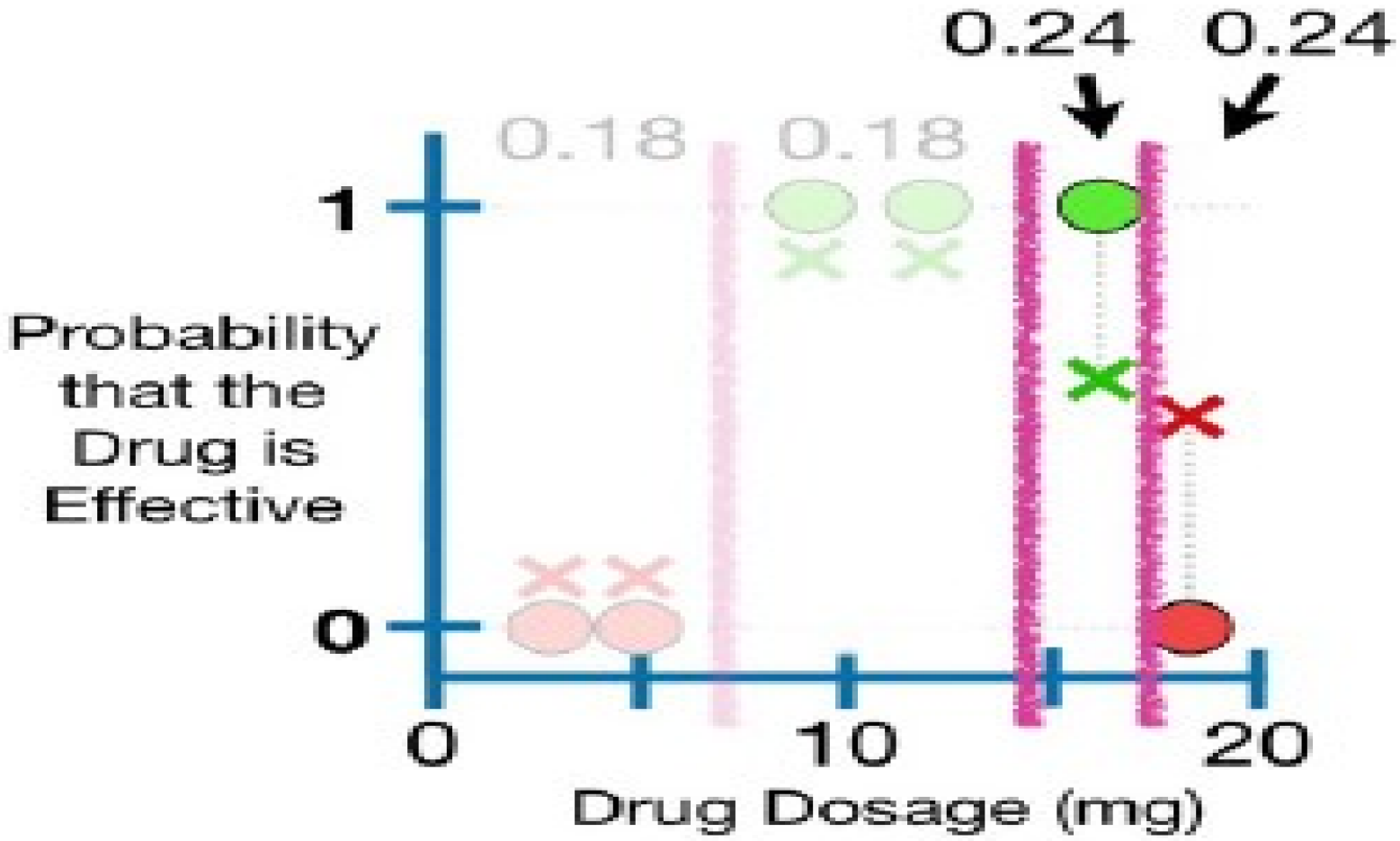
Weighted quantiles focus on uncertain rows

Each row counts by p × (1 – p): confident rows weigh little, uncertain rows weigh more



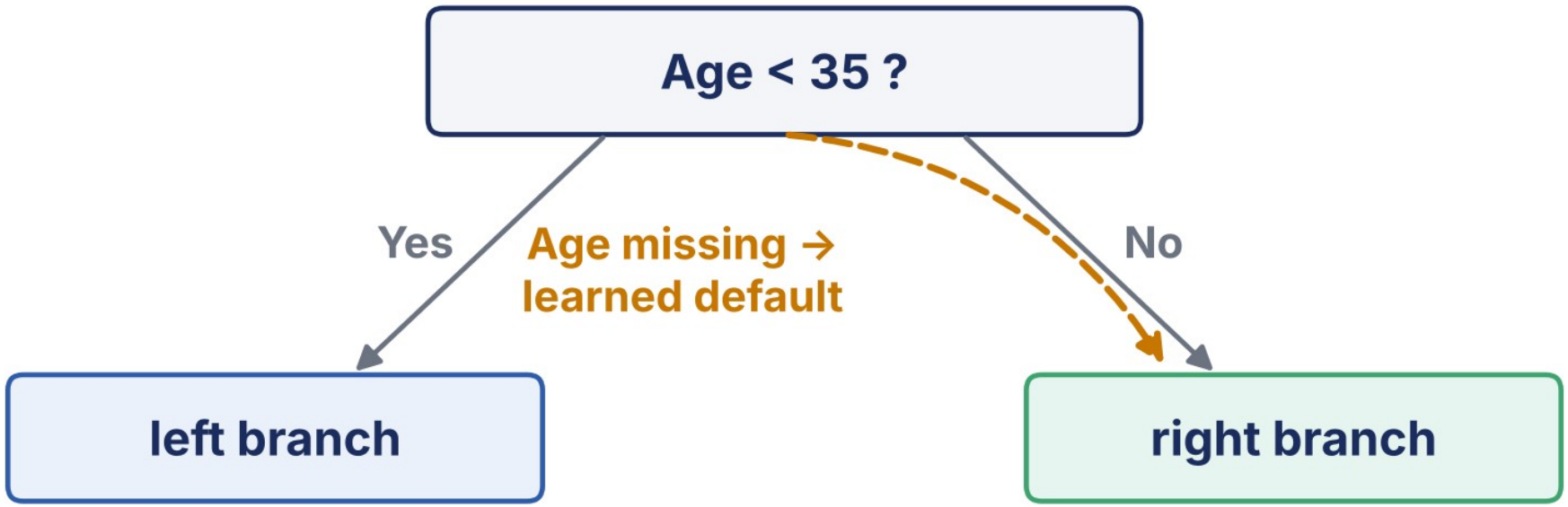
Each weighted quantile holds the same total weight

Uncertain rows weigh 0.24 each, so one of them fills a whole quantile on its own



Missing values follow a learned default direction

Sparsity-aware split finding: no need to impute missing values before training



XGBoost tries sending all missing rows left, then right, and keeps the side with higher gain

Rows with missing values still have residuals

Dosage is missing (???) in two rows; their residuals can still be placed in a leaf

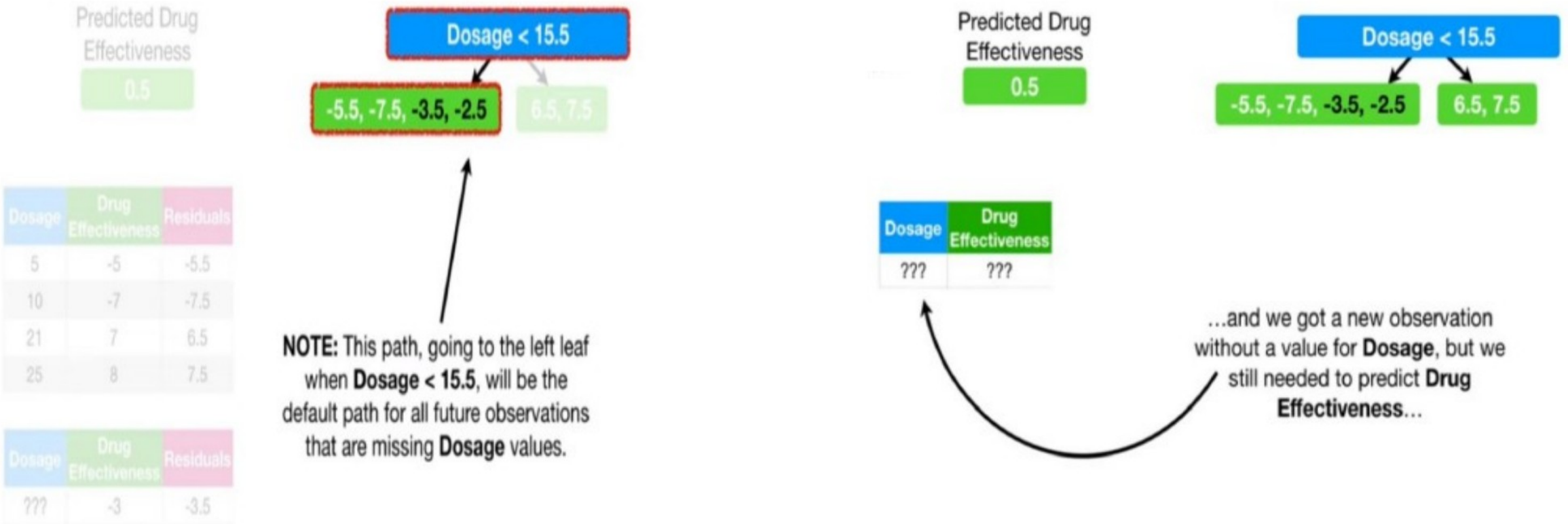


Dosage	Drug Effectiveness	Residuals
10	-7	-7.5
???	-3	-3.5
21	7	6.5
25	8	7.5
5	-5	-5.5
???	-2	-2.5

Dosage	Drug Effectiveness	Residuals
10	-7	-7.5
???	-3	-3.5
21	7	6.5
25	8	7.5
5	-5	-5.5
???	-2	-2.5

New rows with missing values follow the learned path

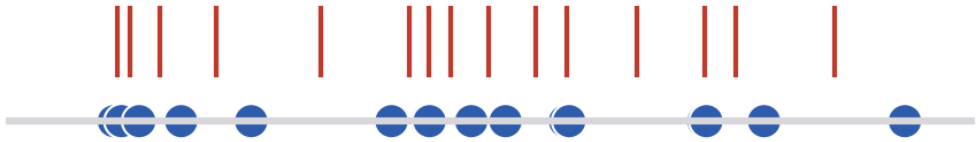
Left: missing rows go to the side with higher gain. Right: new rows use that side too



XGBoost searches fewer cuts and uses every core

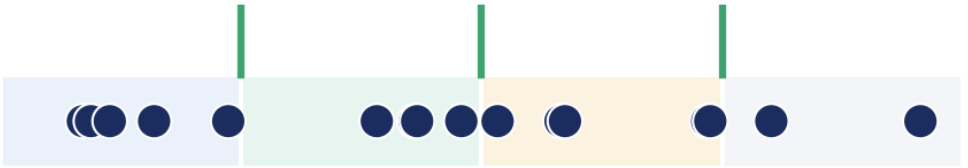
Histogram method: values are grouped into bins first, so each feature has few cuts to test

Classic GBM: try every gap



15 candidate cuts for 16 rows
(1 million rows → about 1 million cuts)

XGBoost: try bin edges only



3 candidate cuts, whatever the row count
(about 256 bins by default)

Classic GBM: one core, one feature at a time



time ≈ 6 blocks

XGBoost: all cores at once

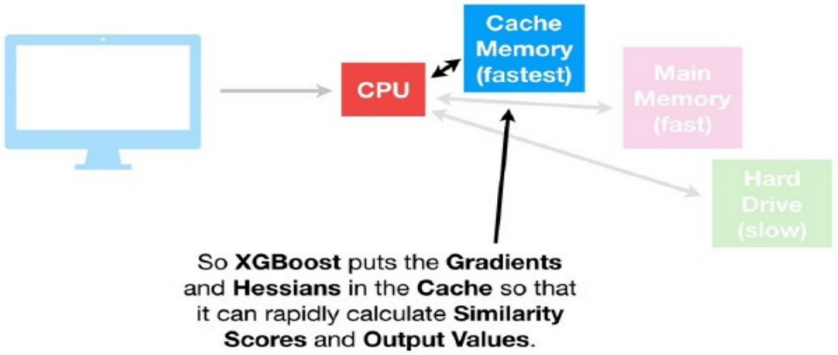


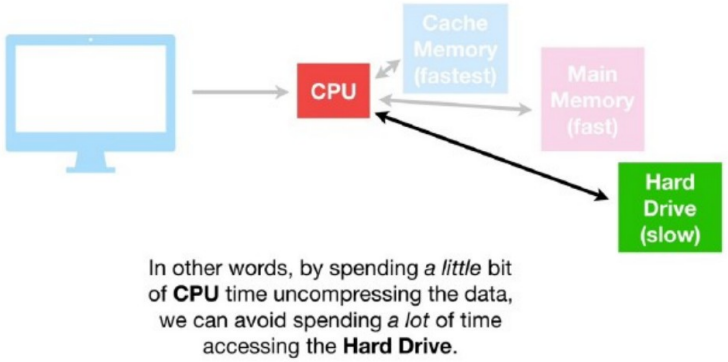
core 1 · core 2 · core 3

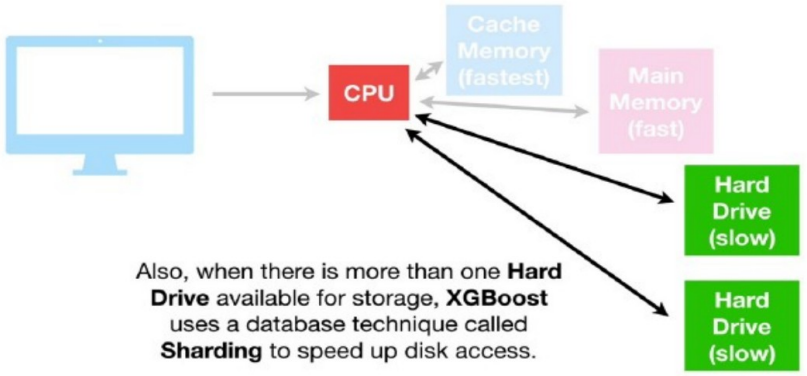
time ≈ 2 blocks

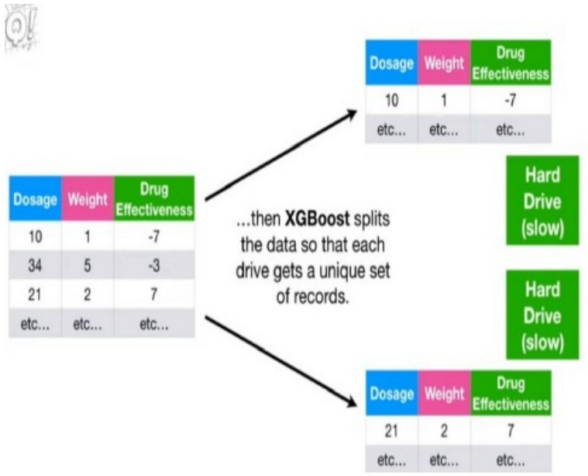
Cache-aware access keeps the CPU busy

Gradients sit in fast cache memory; large data is compressed and split across hard drives







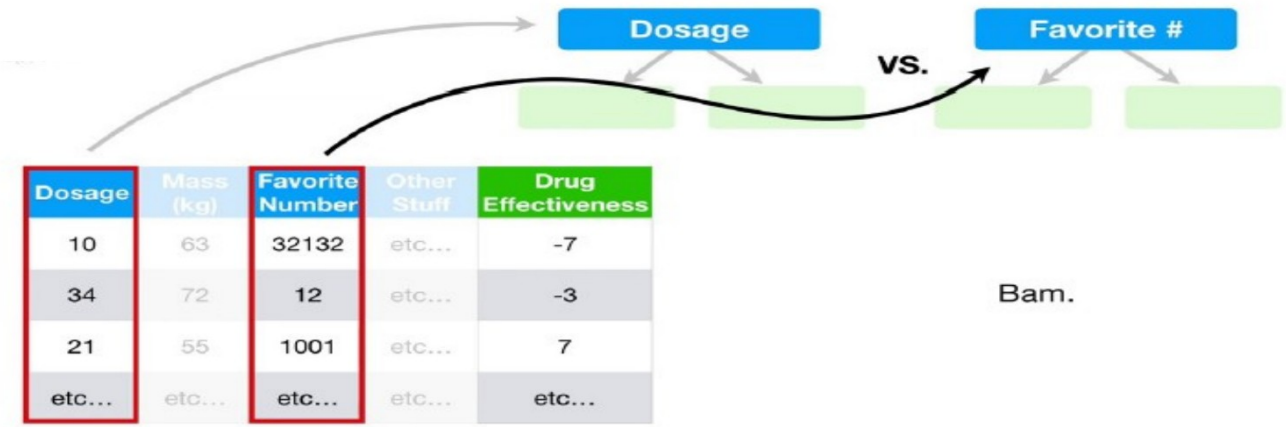


XGBoost also samples rows and columns at random

Like a random forest: each tree sees a random subset, which speeds training and reduces overfitting

Dosage	Mass (kg)	Favorite Number	Other Stuff	Drug Effectiveness
10	63	32132	etc...	-7
34	72	12	etc...	-3
21	55	1001	etc...	7
etc...	etc...	etc...	etc...	etc...

Lastly, I need to mention that **XGBoost** can also speed things up by allowing you to build each tree with only a random subset of the data.

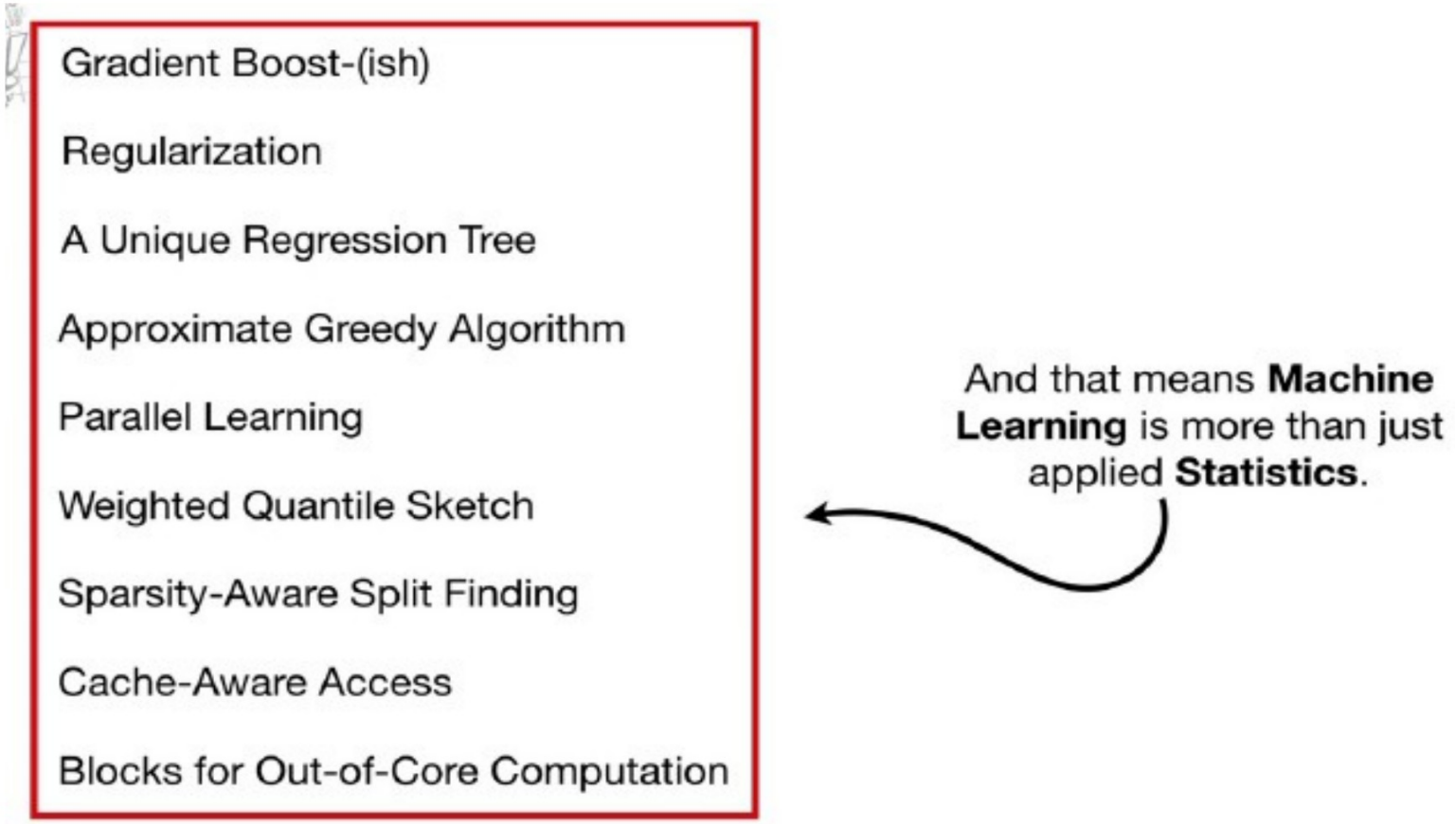


XGBoost is boosting plus a lot of systems engineering

The full list of ideas packed into one library

Take-away

- The statistics: boosting with penalties.
- The engineering: sketches, parallel work, smart memory use.
- Machine learning is more than applied statistics.



Head to head: a bit more accurate, far faster

30,000 rows, 20% of one feature missing, 400 depth-4 trees each; time on this machine



XGBoost became famous through a physics contest

Higgs Boson Machine Learning Challenge on Kaggle, 2014

A student's open-source library spread from one competition to most tabular leaderboards

What made it win

- Fast enough to try many settings.
- Penalties stopped trees from memorizing noise.
- Missing values handled without extra work.

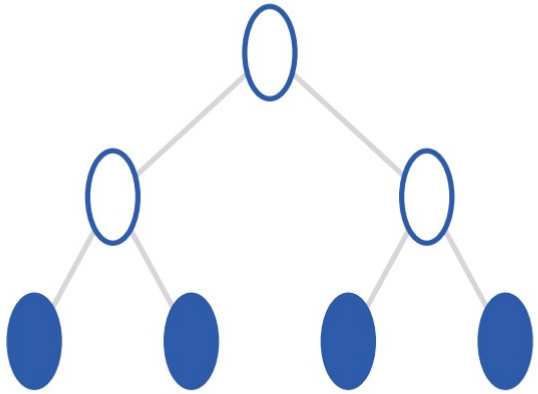
What followed

- Microsoft released LightGBM and Yandex released CatBoost in 2017.
- All three are now standard tools in industry.

The three libraries grow trees in different shapes

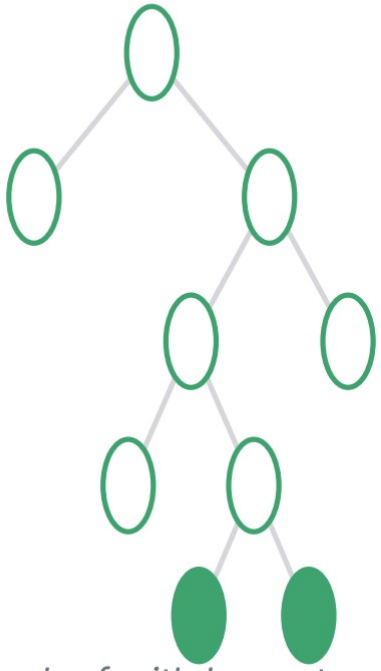
Filled circles = nodes split in the latest step

Level-wise (XGBoost)



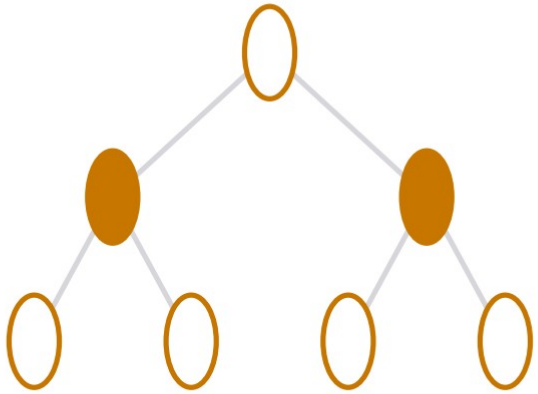
grow a full level at a time

Leaf-wise (LightGBM)



split the leaf with largest gain

Symmetric (CatBoost)

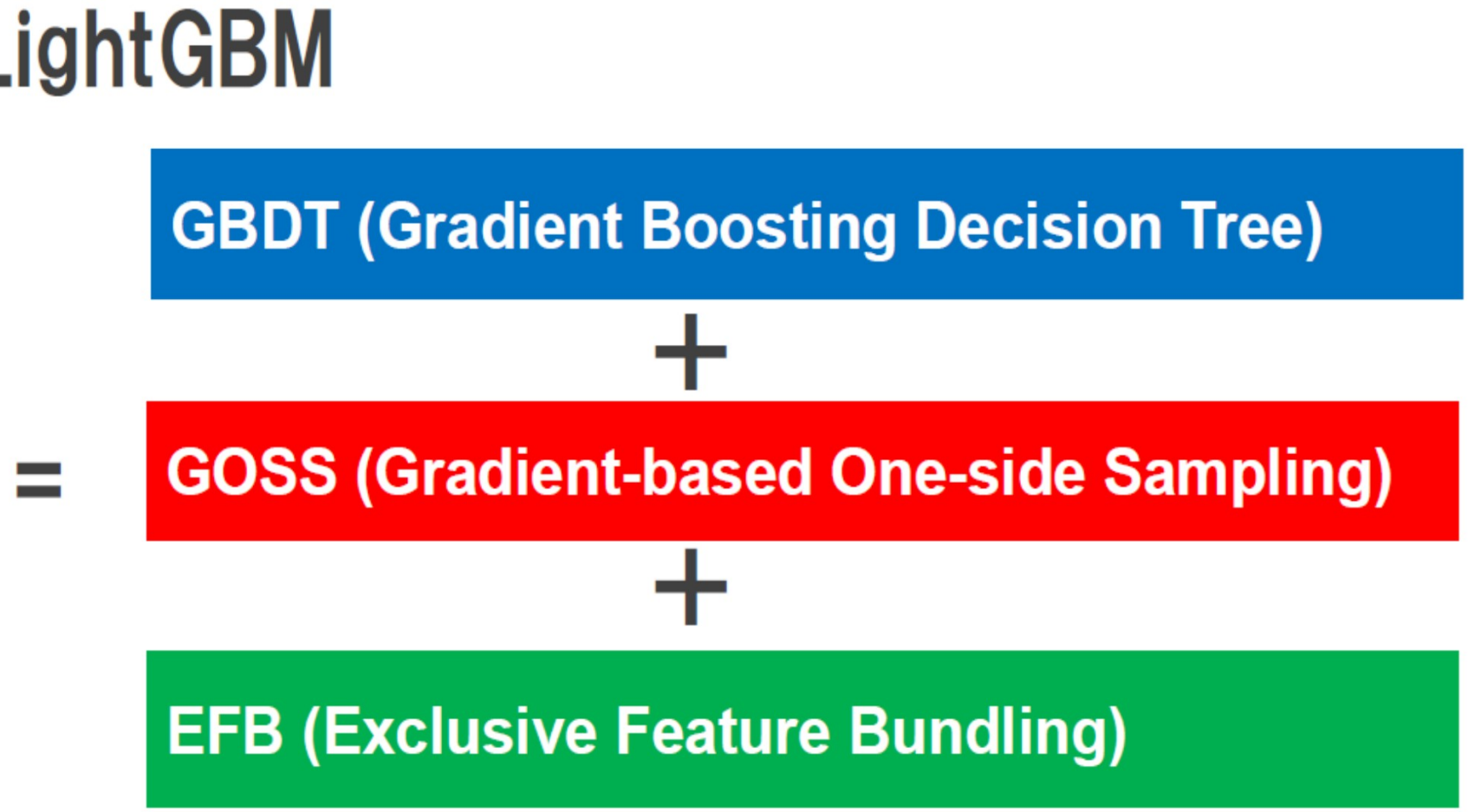


same question: x3 < 0.7 ?

same rule on every node of a level

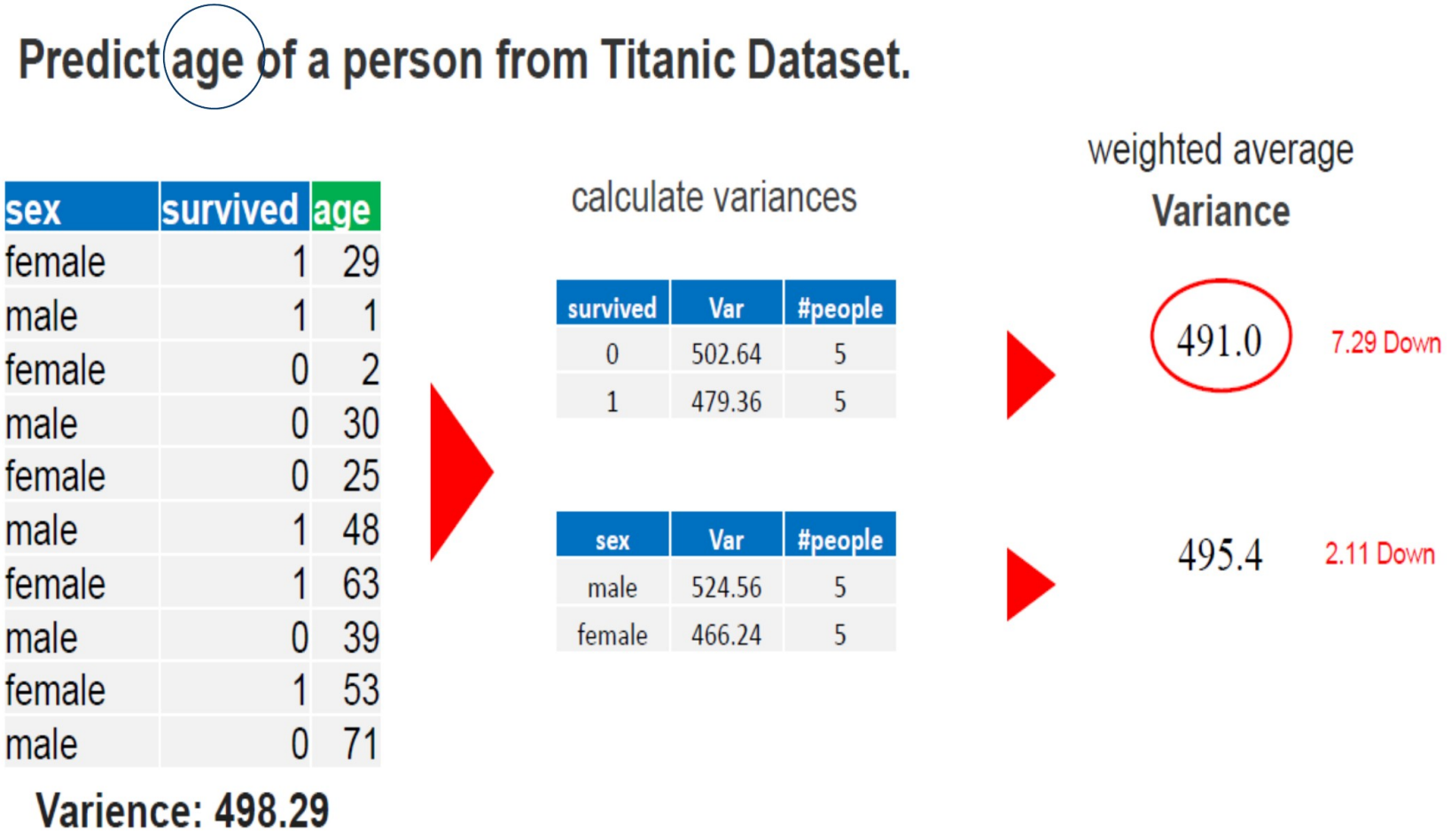
LightGBM adds two tricks on top of gradient boosting

GOSS samples rows smartly; EFB bundles sparse columns



Splits are scored on histograms of the data

Titanic example: predict age; compare the weighted variance left after splitting by survival or by sex



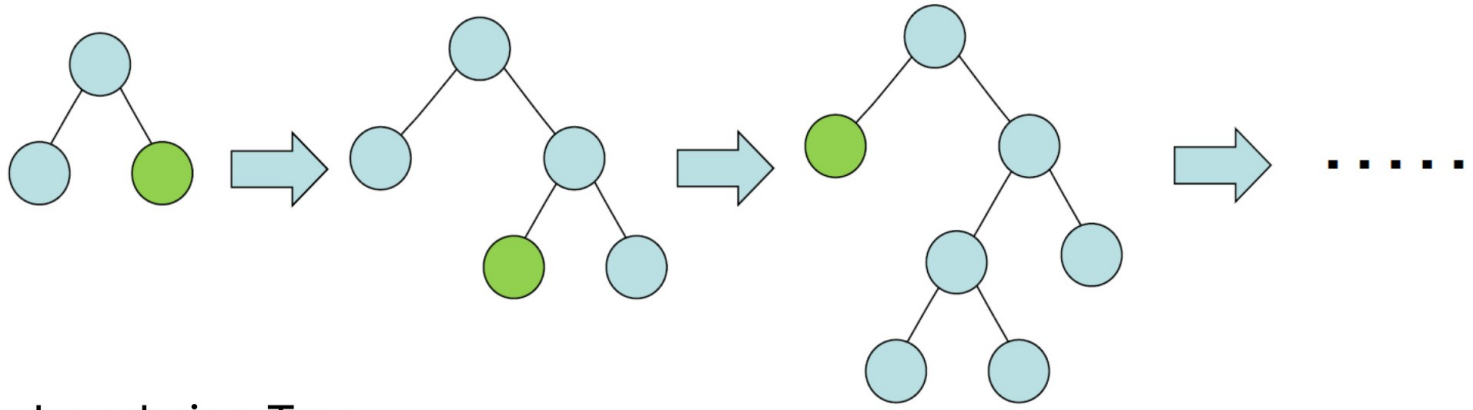
LightGBM grows the most useful leaf first

Top: leaf-wise growth. Bottom: level-wise growth used by XGBoost

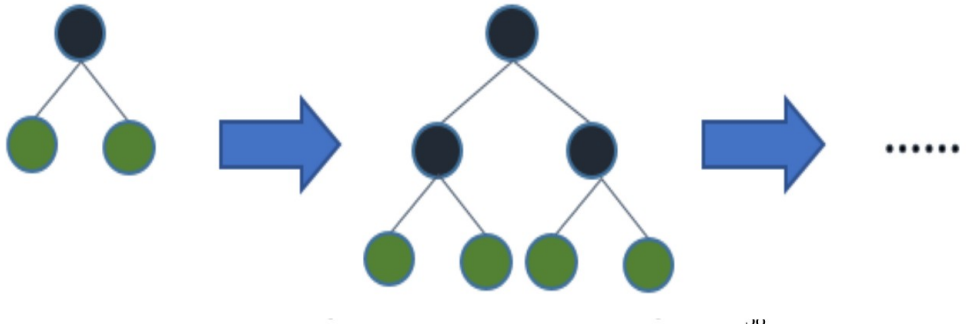
Why it matters

- Leaf-wise trees reach low error with fewer splits.
- They can grow deep and lopsided.
- Limit num_leaves or depth on small data.

- Leafwise Tree (Debthwise Tree)

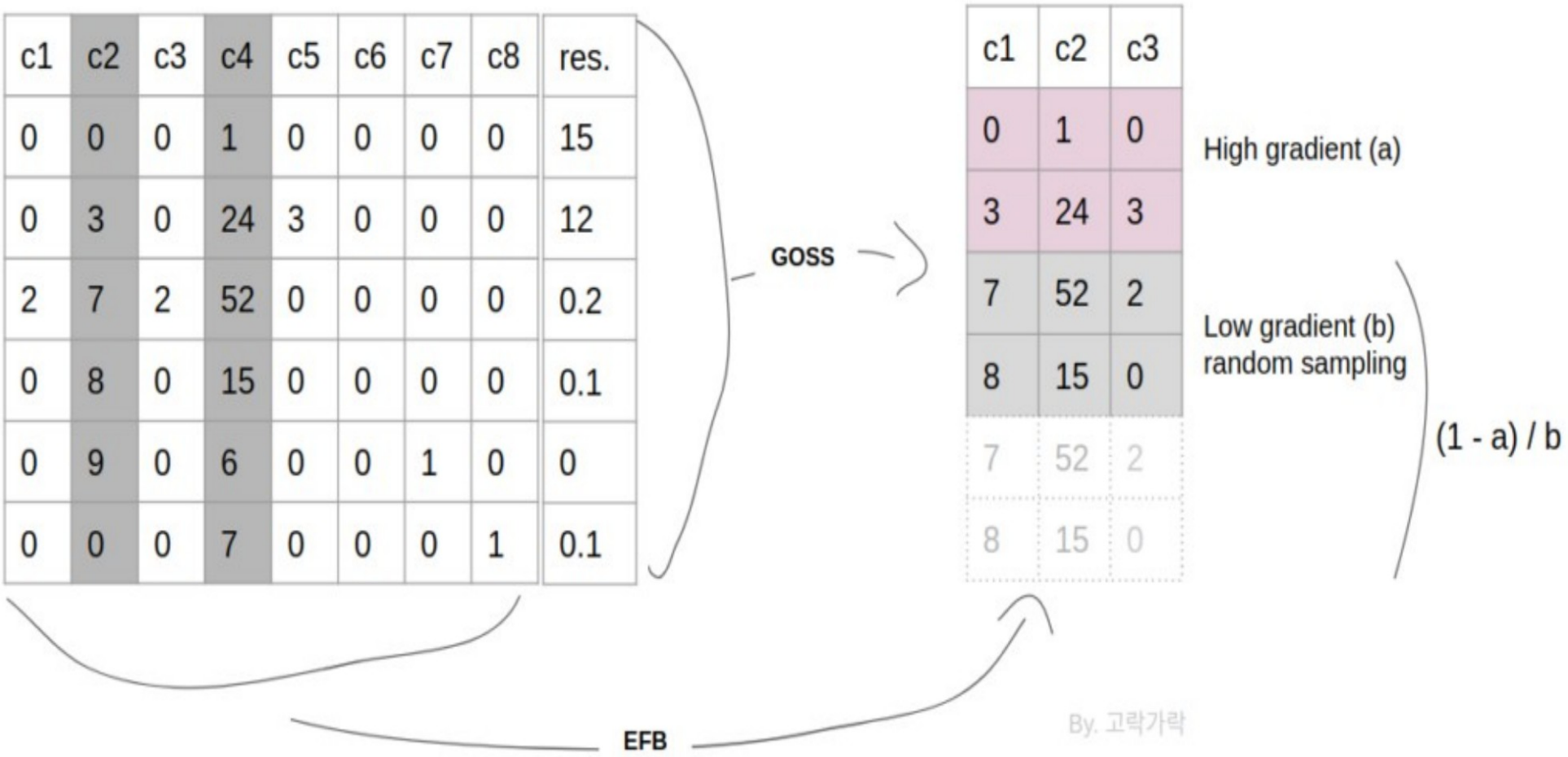


- Levelwise Tree



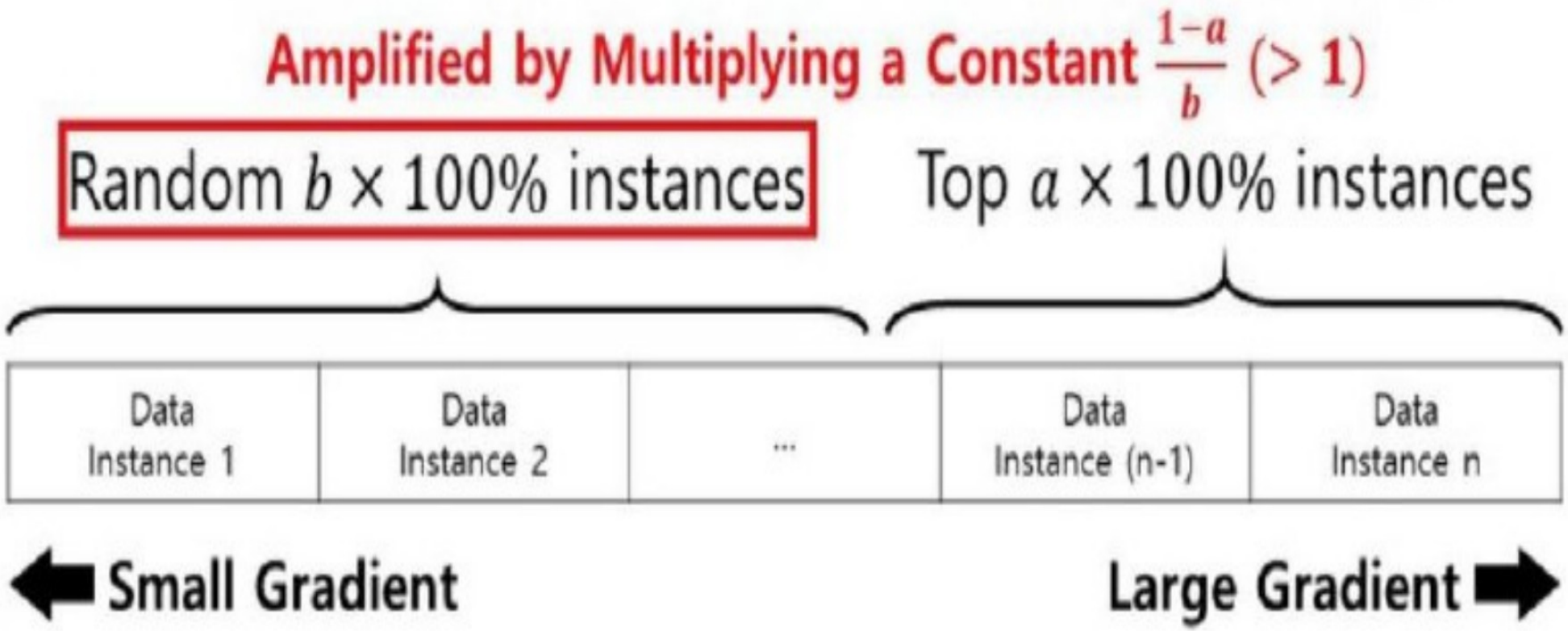
GOSS and EFB shrink the data from two directions

GOSS removes easy rows (fewer rows); EFB merges sparse columns (fewer columns)



GOSS keeps hard rows and samples easy ones

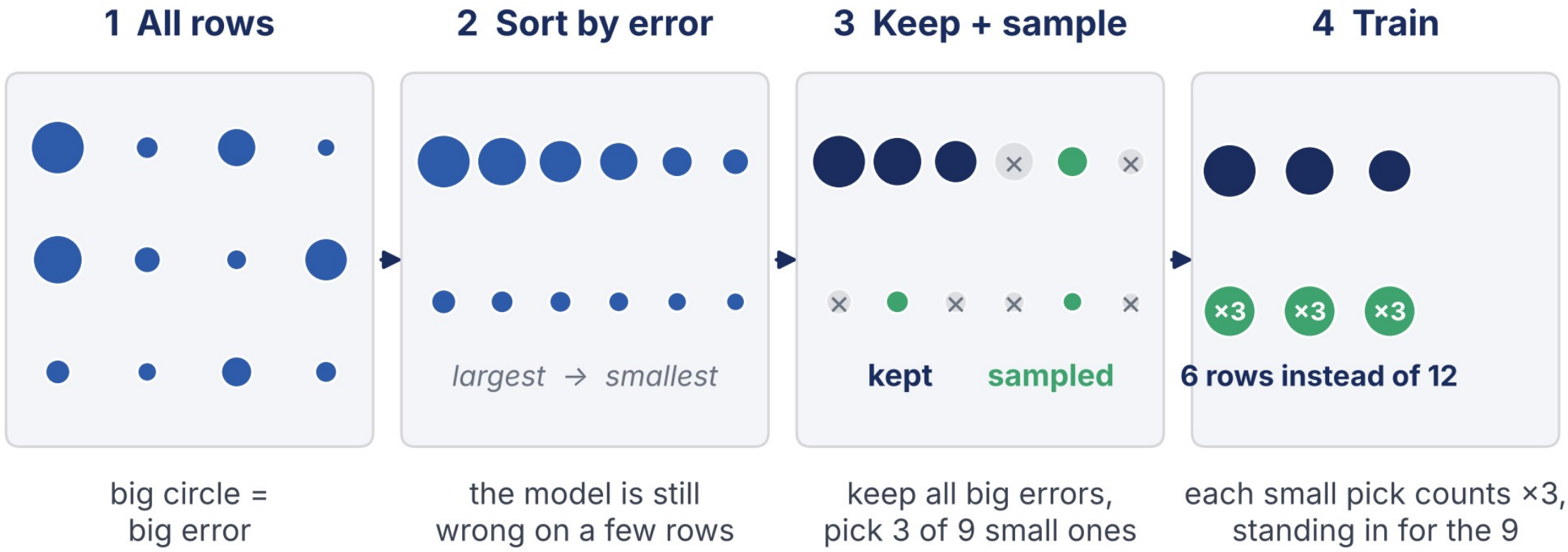
Sort rows by gradient size: keep the top a × 100%, sample b × 100% of the rest, and scale those up



https://cdm98.tistory.com/m/31

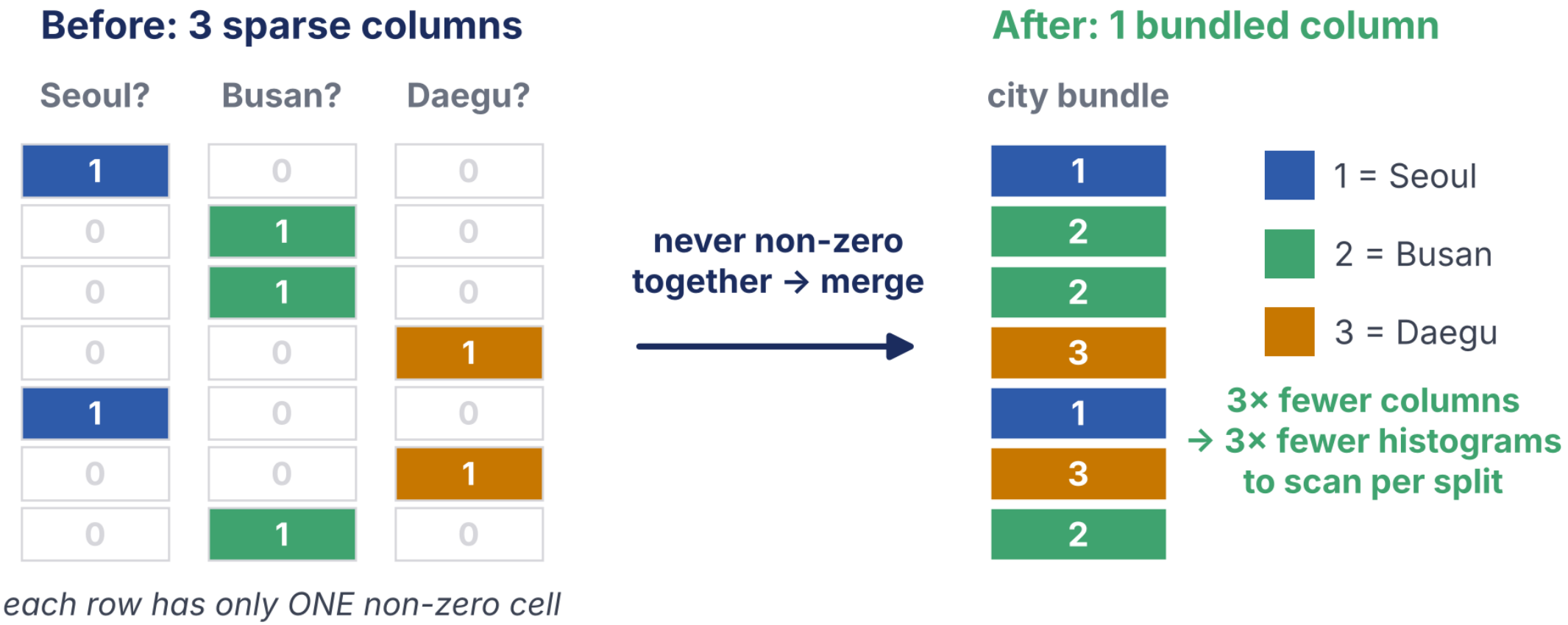
LightGBM trains on the rows that still have big errors

GOSS = Gradient-based One-Side Sampling: keep the hard rows, sample the easy ones



LightGBM merges columns that are never on together

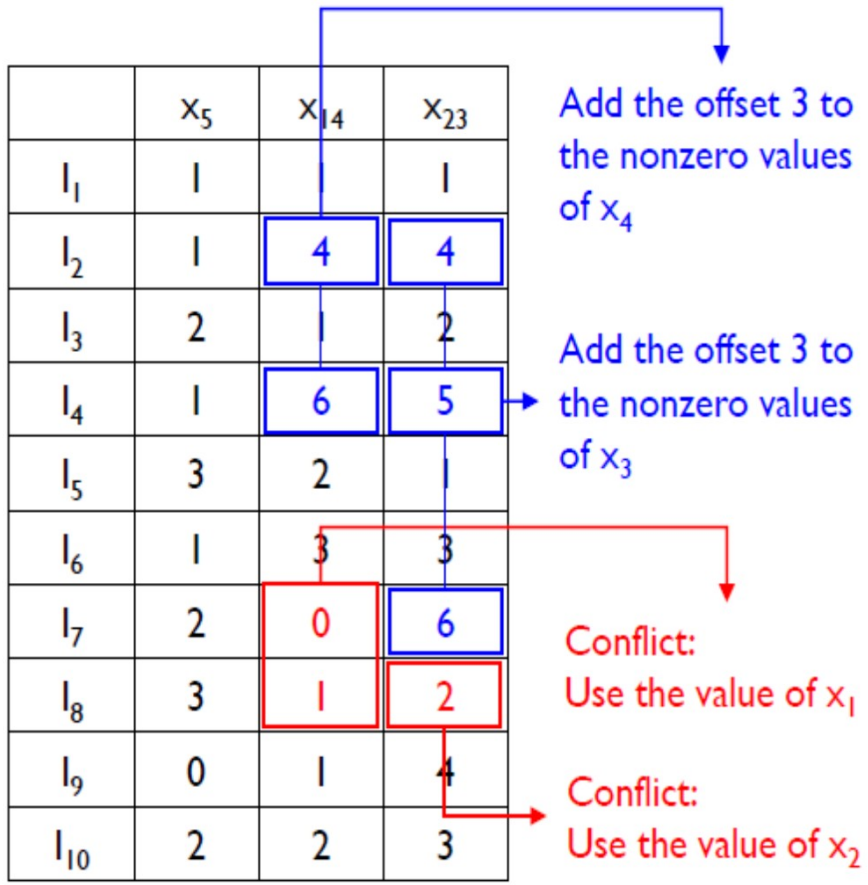
EFB = Exclusive Feature Bundling: sparse one-hot columns become one column



EFB adds an offset so bundled values stay apart

Original example: values of x4 are shifted by 3 before merging with x1; rare conflicts keep one value

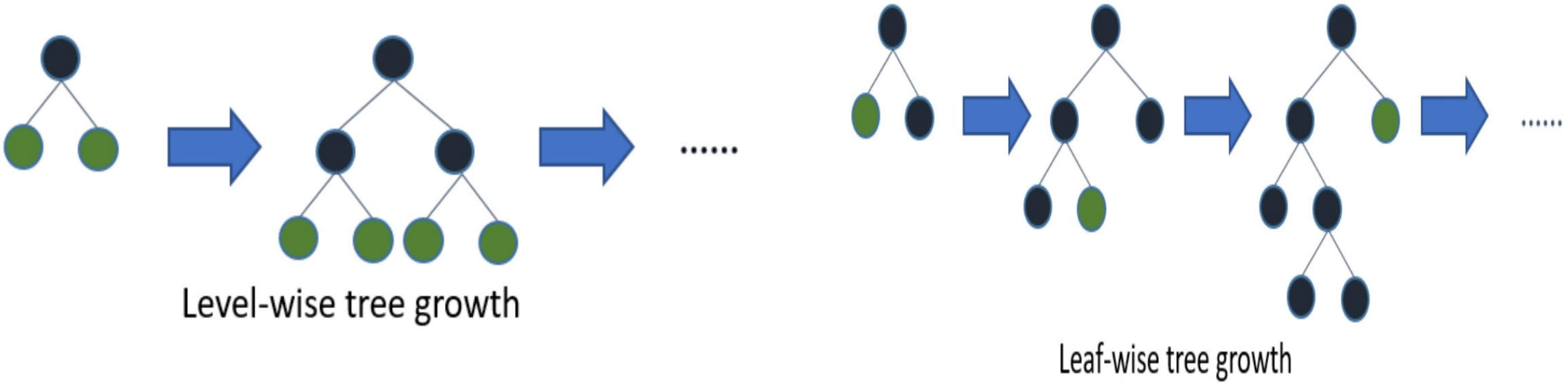
	x_5	x_1	x_4	x_2	x_3
I_1	1	1	0	1	0
I_2	1	0	1	0	1
I_3	2	1	0	2	0
I_4	1	0	3	0	2
I_5	3	2	0	1	0
I_6	1	3	0	3	0
I_7	2	0	0	0	3
I_8	3	1	4	2	3
I_9	0	1	0	0	1
I_{10}	2	2	0	3	0



	x_5	x_{14}	x_{23}
I_1	1	1	1
I_2	1	4	4
I_3	2	1	2
I_4	1	6	5
I_5	3	2	1
I_6	1	3	3
I_7	2	0	6
I_8	3	1	2
I_9	0	1	4
I_{10}	2	2	3

CatBoost builds level-wise, symmetric trees

Left: level-wise growth (XGBoost, CatBoost). Right: leaf-wise growth (LightGBM)



Plain target encoding leaks the answer

Predict the day's highest temperature from the weather category

The problem

- Replace “cloudy” by the average temperature of cloudy days: 18.5.
- That average already contains each row's own answer.
- The model learns from a value it will not have at prediction time.

time	feature 1	class_labels (max_tempetarure on that day)
sunday	sunny	35
monday	sunny	32
tues	couldy	15
wed	cloudy	14
thurs	mostly_cloudy	10
fri	cloudy	20
sat	cloudy	25

CatBoost encodes categories using only earlier rows

Ordered target statistics: a row never sees its own target, which prevents target leakage

row	color	target	earlier rows, same color	encoded value
1	Red	1	none yet	0.50
2	Blue	0	none yet	0.50
3	Red	0	targets 1	0.75
4	Red	1	targets 1, 0	0.50
5	Blue	1	targets 0	0.25
6	Red	1	targets 1, 0, 1	0.62

CatBoost combines categories that move together

Ordered feature combination: country and hair color tell the same story here

What it does

- Country alone already decides hair color.
- CatBoost tests combined categories during training.
- Users do not need to build these combinations by hand.

country	hair color	class_label
India	black	1
India	black	1
India	black	1
india	black	1
russia	white	0
russia	white	0
russia	white	0
russia	white	0

Each library has a different default strength

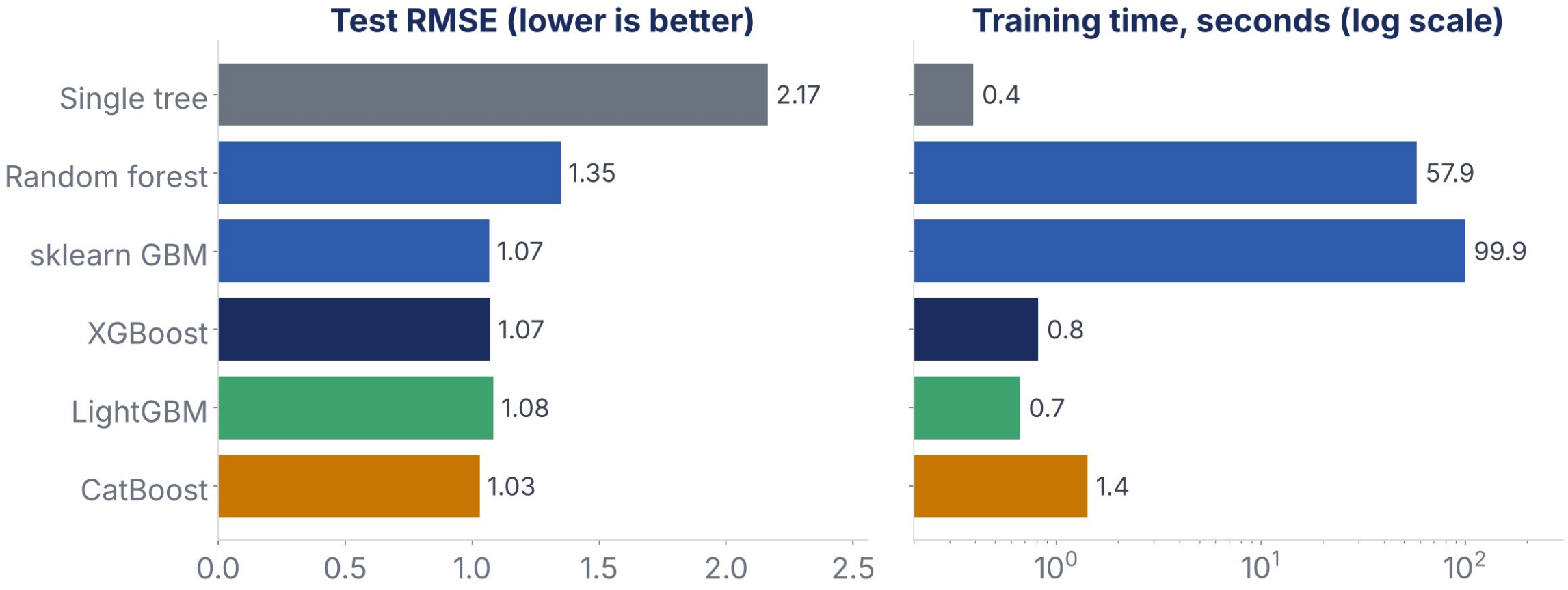
Typical defaults; all three can be tuned to behave alike

	XGBoost	LightGBM	CatBoost
Tree growth	level-wise	leaf-wise	symmetric
Missing values	learned direction	learned direction	built-in handling
Categorical data	needs encoding*	native support	ordered statistics
Best known for	robust all-rounder	speed on large data	little tuning needed

* recent XGBoost versions add experimental native categorical support

Modern libraries match classic boosting, far faster

Friedman simulated data, 30,000 training rows, 500 trees; times depend on the machine



Tree ensembles still lead on tabular data

Grinsztajn, Oyallon and Varoquaux (NeurIPS 2022) compared models on 45 data sets

On medium-sized tables, tuned tree ensembles beat deep neural networks on most data sets

Why trees win on tables

- Real targets change sharply, not smoothly.
- Many columns are uninformative; trees ignore them.
- No scaling or careful preprocessing needed.

Where deep nets win

- Images, audio and text, where raw inputs have structure.
- Very large data sets, and mixed inputs with text fields.

05

PART 5 · EXPLAINING TREE MODELS

Feature Importance and SHAP

Which inputs does the model use, and why did it make this prediction?

Feature importance shows which inputs matter

Three common tools, each answering a slightly different question

Split-based

built into trees

How much did splits on this feature clean up the nodes?

fast, but biased

Permutation

shuffle and re-test

How much worse does the model get if this feature is scrambled?

honest, global only

SHAP

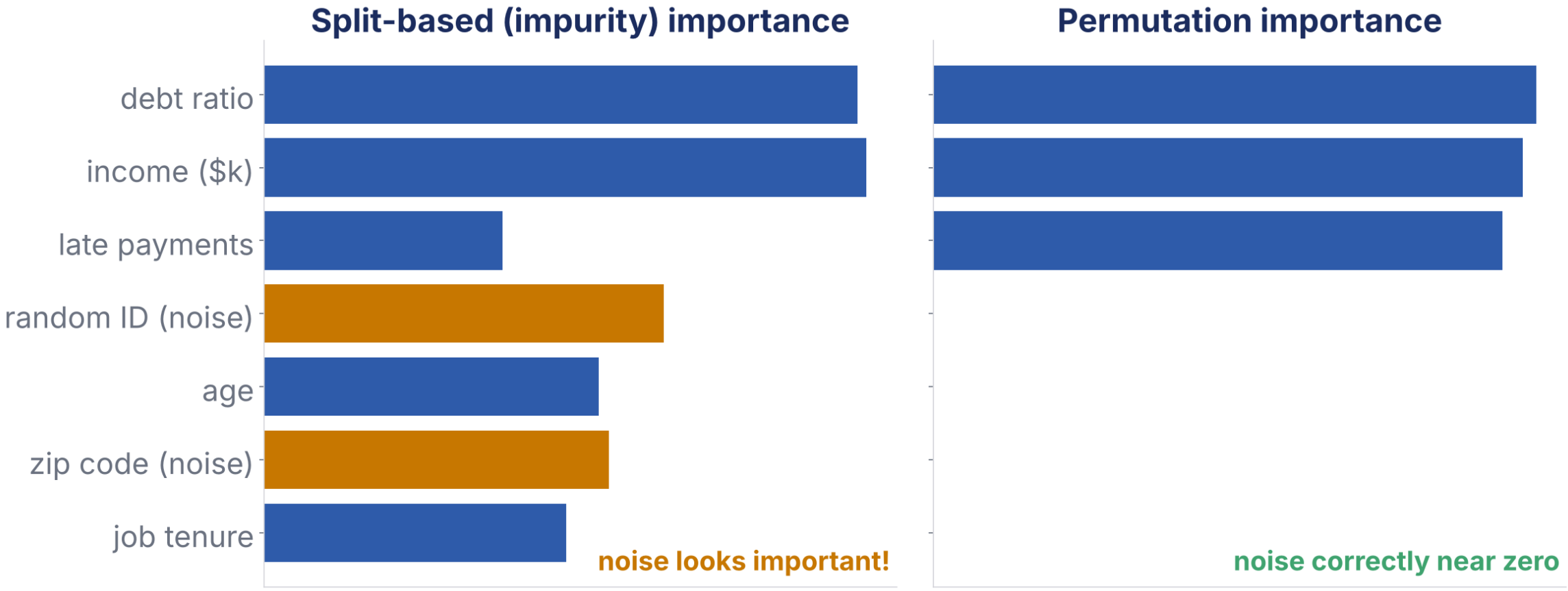
fair credit per row

How much did this feature push this one prediction up or down?

global and local

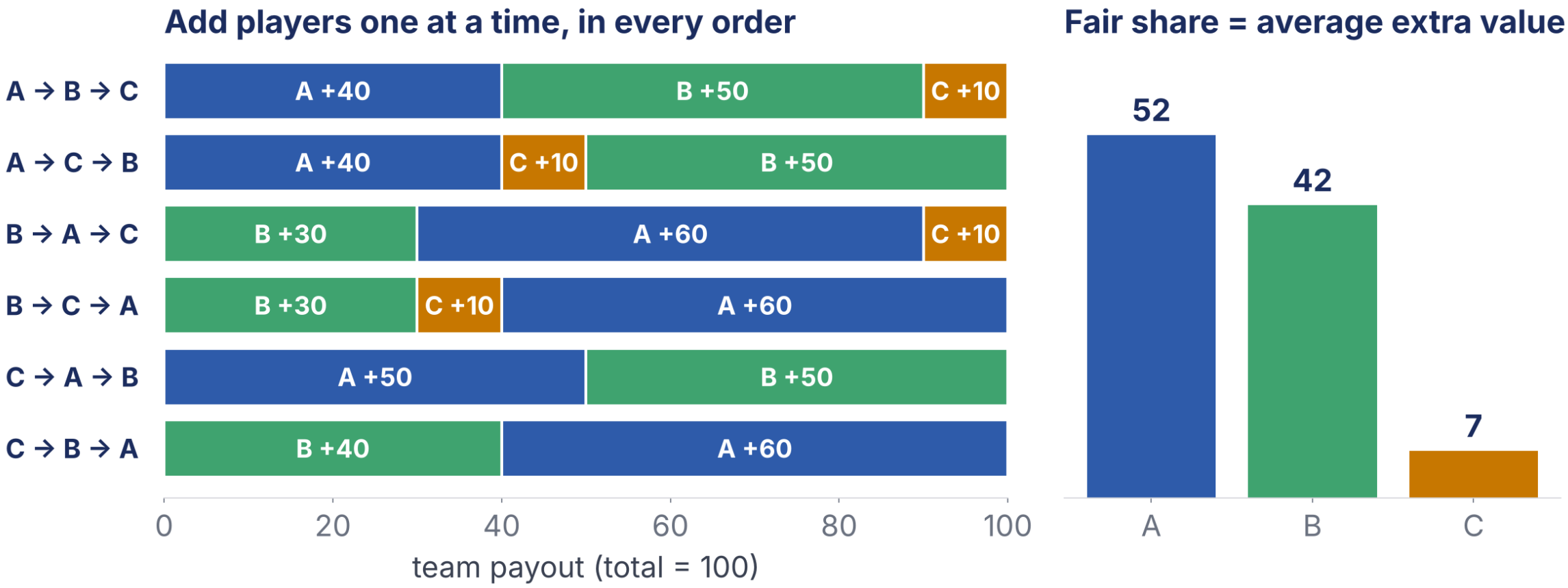
Split-based importance can be fooled by noise

Simulated loan data with two pure-noise columns; importance measured on held-out rows



Shapley values split a team payout fairly

Game theory (Lloyd Shapley, Nobel 2012): average each player's extra value over every joining order



SHAP treats features as players in a team

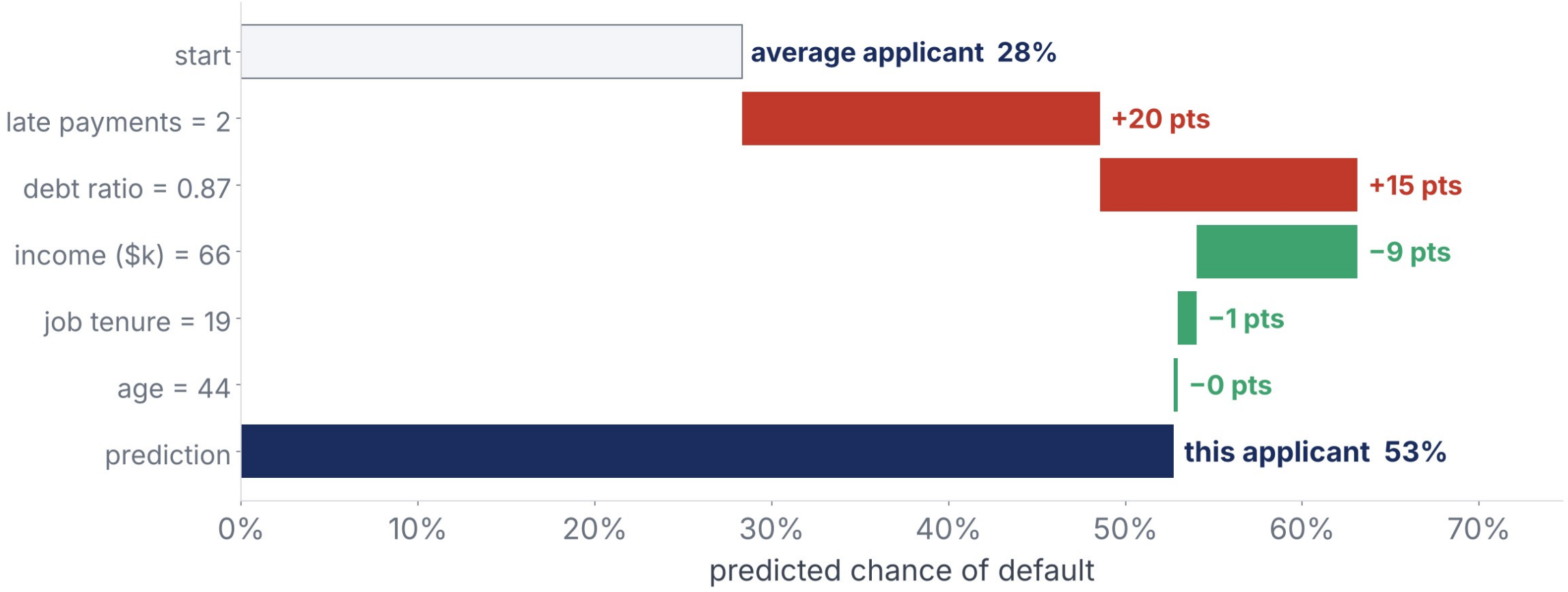
SHAP = SHapley Additive exPlanations (Lundberg and Lee, 2017)

In the team game	In the tree model
players	features of one applicant
team payout	prediction minus the average prediction
fair share of each player	SHAP value of each feature
shares add up to the payout	SHAP values add up to the prediction

For tree models, TreeSHAP computes these shares exactly and fast

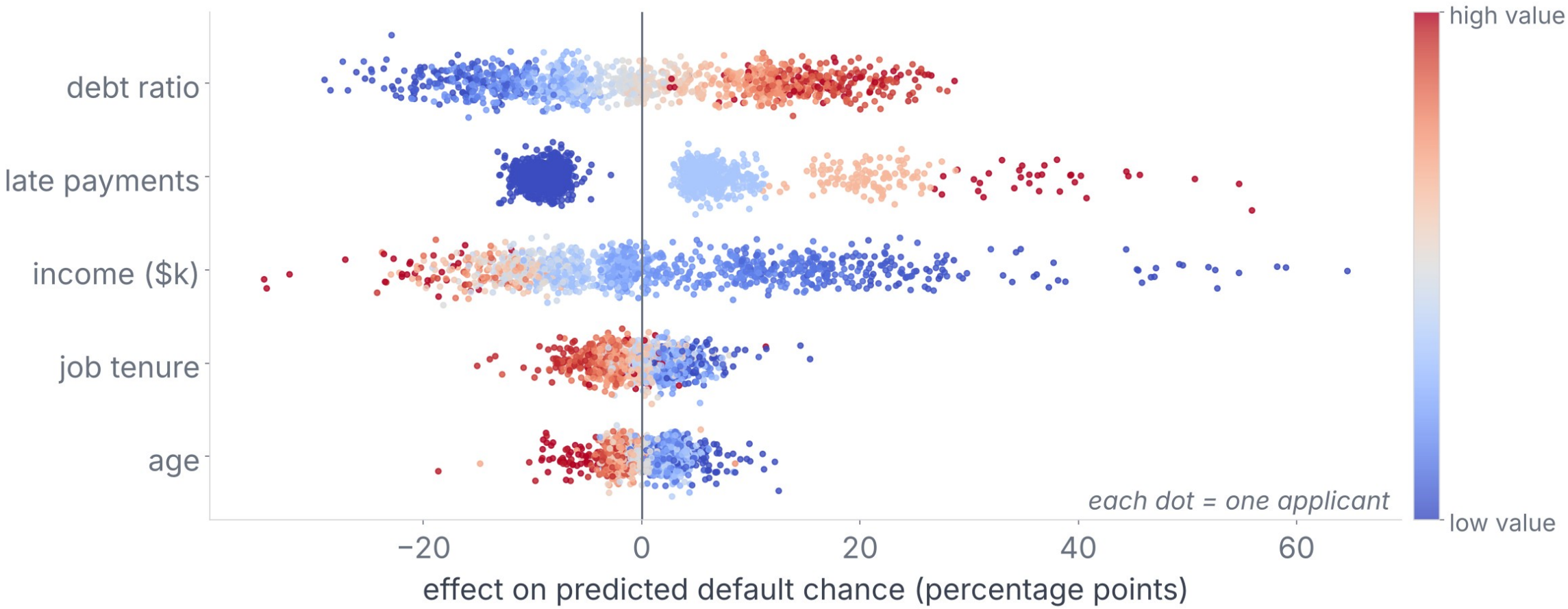
SHAP explains one prediction, feature by feature

One loan applicant: start from the average default chance, then add each feature's push



The summary plot shows size and direction together

Each dot is one applicant; red = high feature value, blue = low; right = higher risk

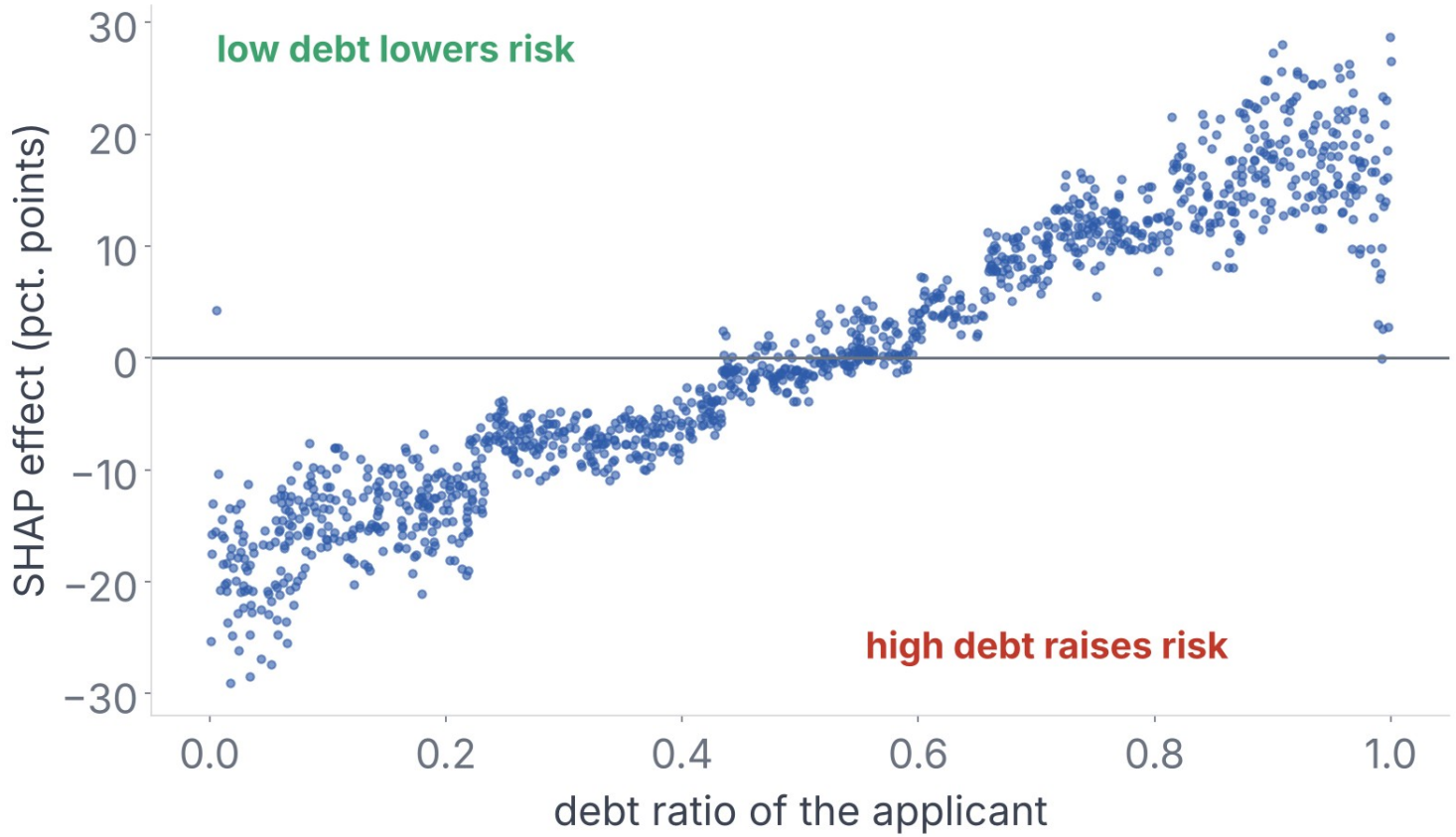


A dependence plot shows how one feature moves risk

Debt ratio against its SHAP effect, one dot per applicant

What we see

- Below about 0.5, debt lowers risk.
- Above it, risk climbs steadily.
- The model learned this shape on its own.



Lenders must explain why they say no

U.S. lenders send adverse action notices that list the main reasons for a rejection

SHAP values can turn a model's decision into a ranked list of reasons for each applicant

How it is used

- Rank features by their SHAP push toward rejection.
- Report the top few as reason codes.
- Check that no unfair feature drives decisions.

Limits to remember

- SHAP explains the model, not the real world.
- Correlated features can share credit in odd ways.
- It is not proof of cause and effect.

A simple recipe works for most tabular problems

Start simple, add complexity only when validation error improves

1 Baseline

random forest

Few settings to tune; check OOB error.

300–500 trees

2 Boost

any modern GBDT

Small learning rate with early stopping.

rate 0.05–0.1

3 Tune

complexity first

Depth or leaves, min child size, row and column subsampling.

cross-validation

4 Explain

before deploying

Permutation importance or SHAP values.

check for leakage

Questions to discuss

No single right answer; justify your choice

Q1	A bank must explain each rejection. Would you use one small tree, or a boosted model with SHAP?	**interpretability**
Q2	Your forest has 1% training error and 20% test error. Which settings would you change first?	**overfitting**
Q3	Why does boosting overfit as trees are added, while a random forest does not?	**bias vs variance**
Q4	Your data has a 5,000-level category (store ID). Which library would you try first, and why?	**categorical data**

Trees are simple; their ensembles are powerful

What to remember

1

Trees split data into boxes

Easy to read, but high variance: limit depth or average many trees.

2

Forests average, boosting corrects

Random forests cut variance; boosting cuts bias with small trees and a small learning rate.

3

SHAP opens the black box

Shapley values give each feature fair credit for one prediction, and add up to a global picture.

Next: hands-on lab with XGBoost and SHAP